

# SQL

Parte IV: Consultas de estructura típicas

Consultas para mostrar el mayor/menor de todos

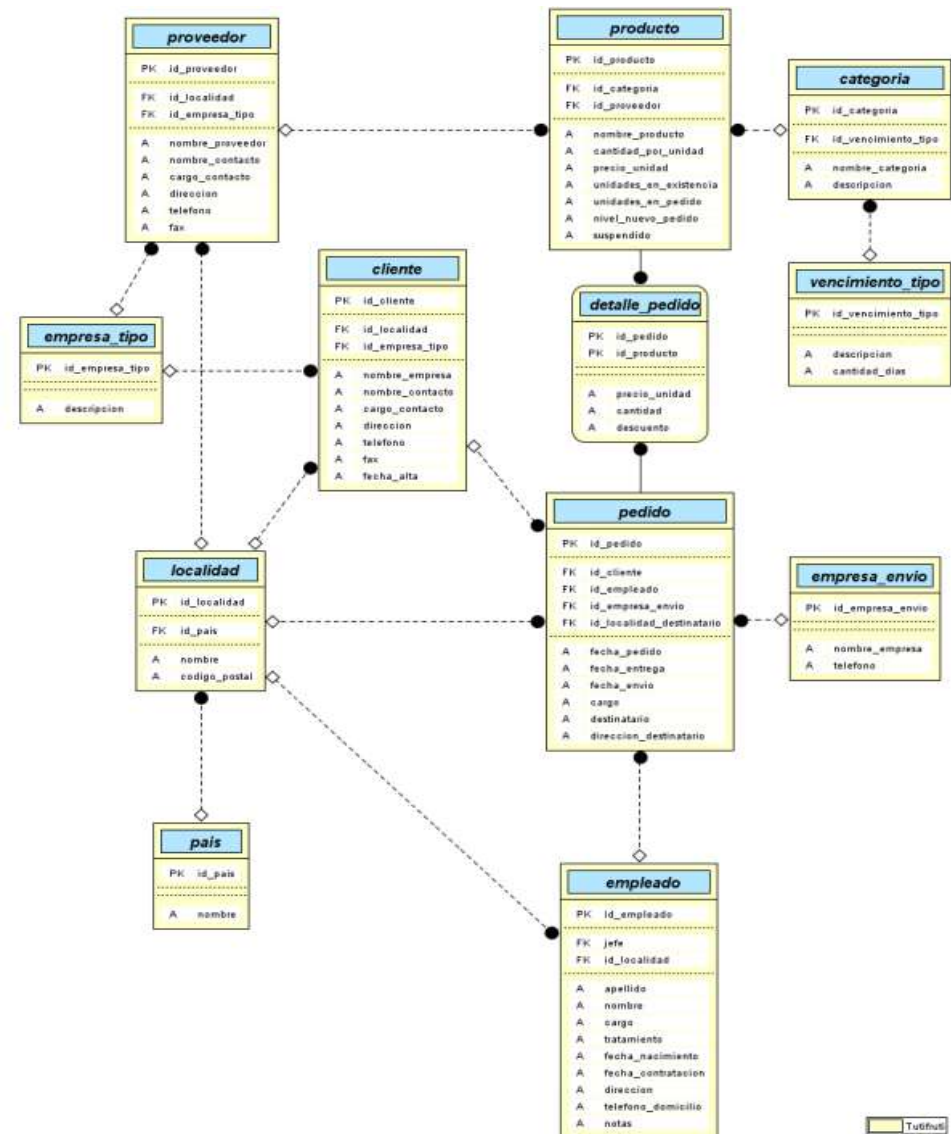
Consultas para responder al “para todos”

Consultas para responder “sólo”

Encontrar el Mayor/Menor de todos en un conjunto

“Encontrar la categoría de productos, que posee la mayor cantidad de productos”.

Select c.id\_categoria  
From categoría c join producto p on (c.id\_categoria = p.id\_categoria)



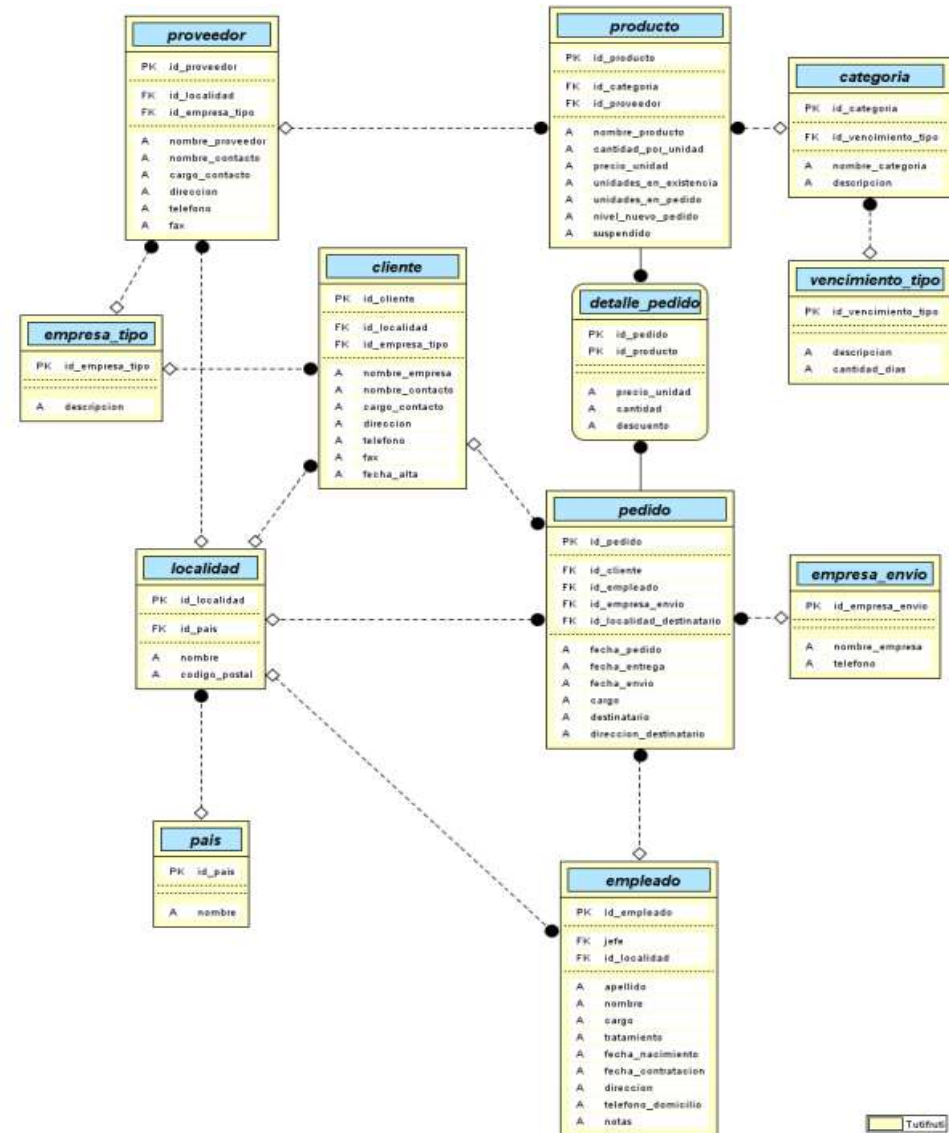
## Encontrar el Mayor/Menor de todos en un conjunto

“Encontrar la categoría de productos, que posee la mayor cantidad de productos”.

Select c.id\_categoría

From categoría c join producto p on (c.id\_categoría = p.id\_categoría)

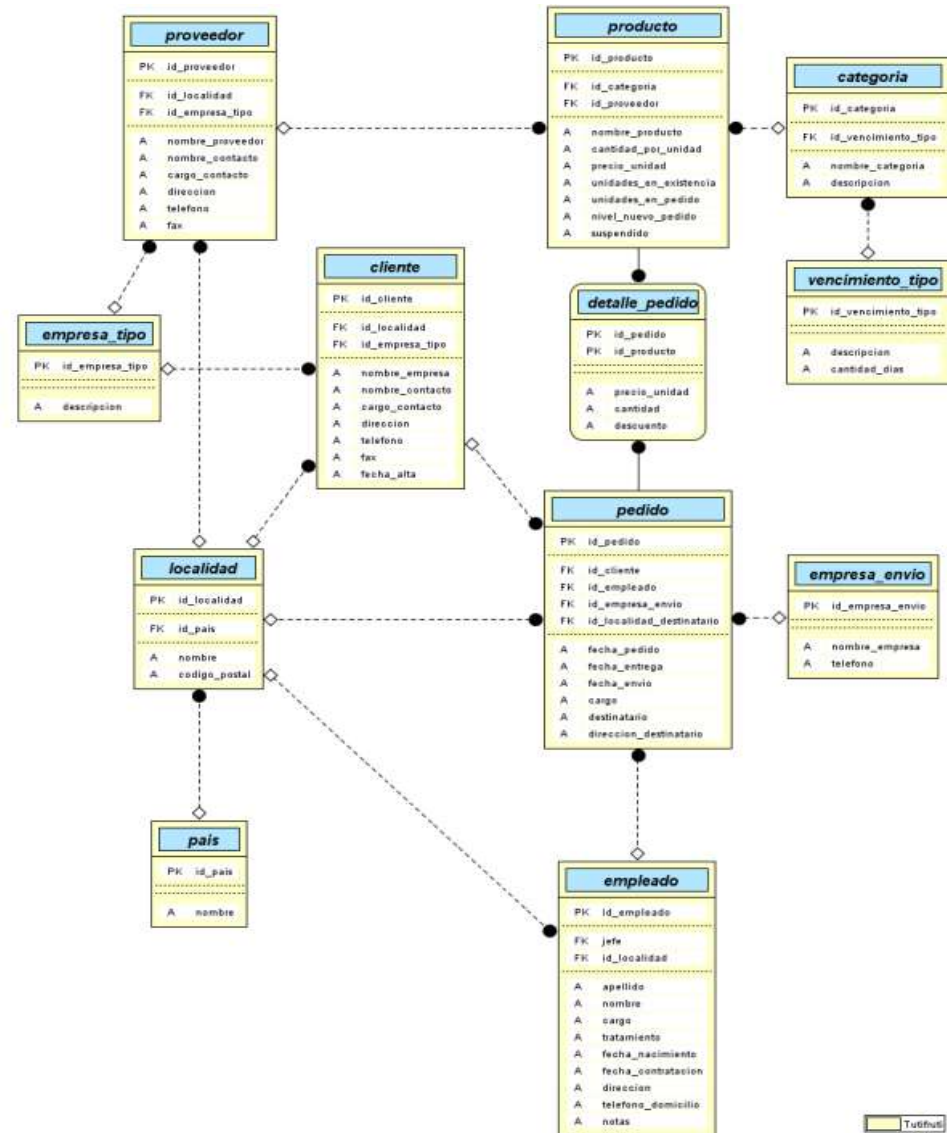
Group by c.id\_categoría



## Encontrar el Mayor/Menor de todos en un conjunto

“Encontrar la categoría de productos, que posee la mayor cantidad de productos”.

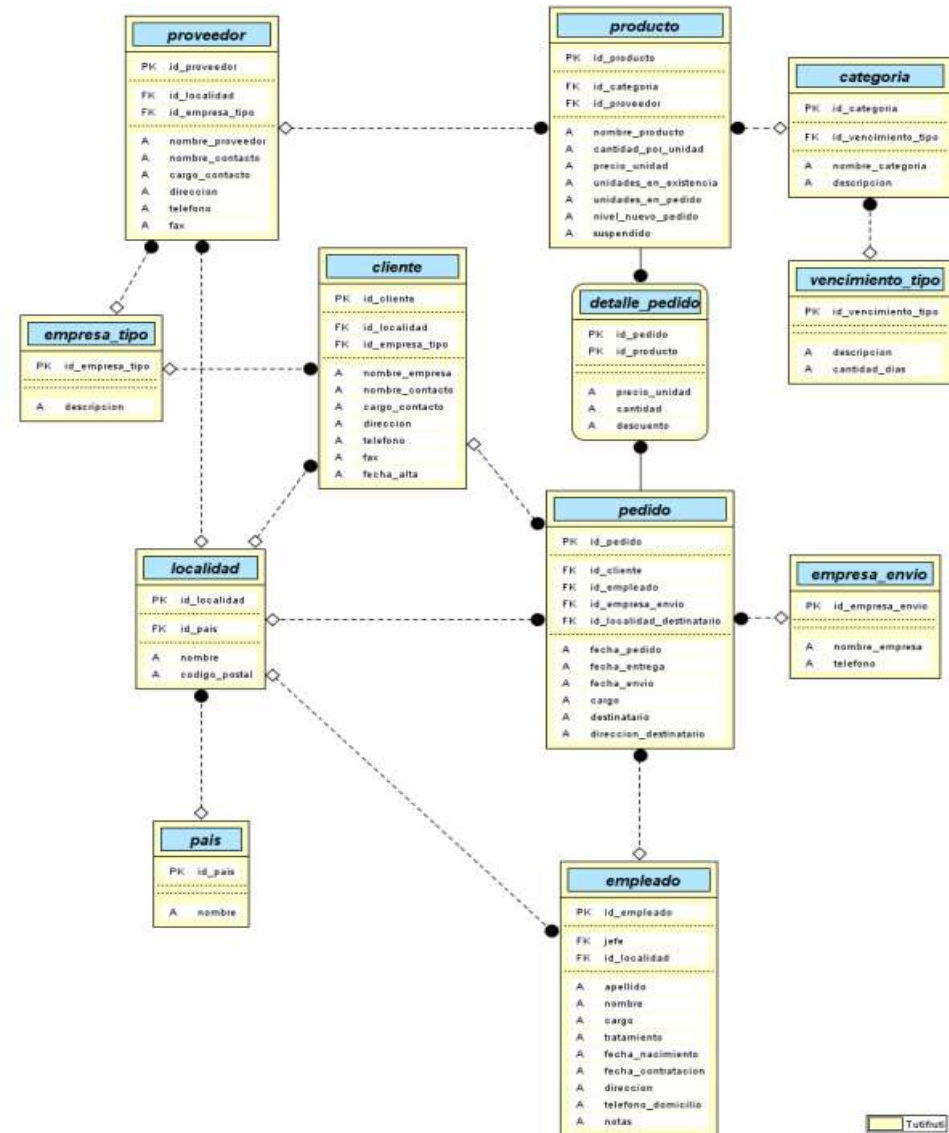
```
Select c.id_categoria
From categoría c join producto p on (c.id_categoria = p.id_categoria)
Group by c.id_categoria
Having count(*)
```



## Encontrar el Mayor/Menor de todos en un conjunto

“Encontrar la categoría de productos, que posee la mayor cantidad de productos”.

Select c.id\_categoria  
 From categoría c join producto p on (c.id\_categoria = p.id\_categoria)  
 Group by c.id\_categoria  
 Having count(\*) >>>> (Select c1.id\_categoria  
 From categoría c1 join producto p1 on  
 (c1.id\_categoria = p1.id\_categoria)  
 Group by c1.id\_categoria)

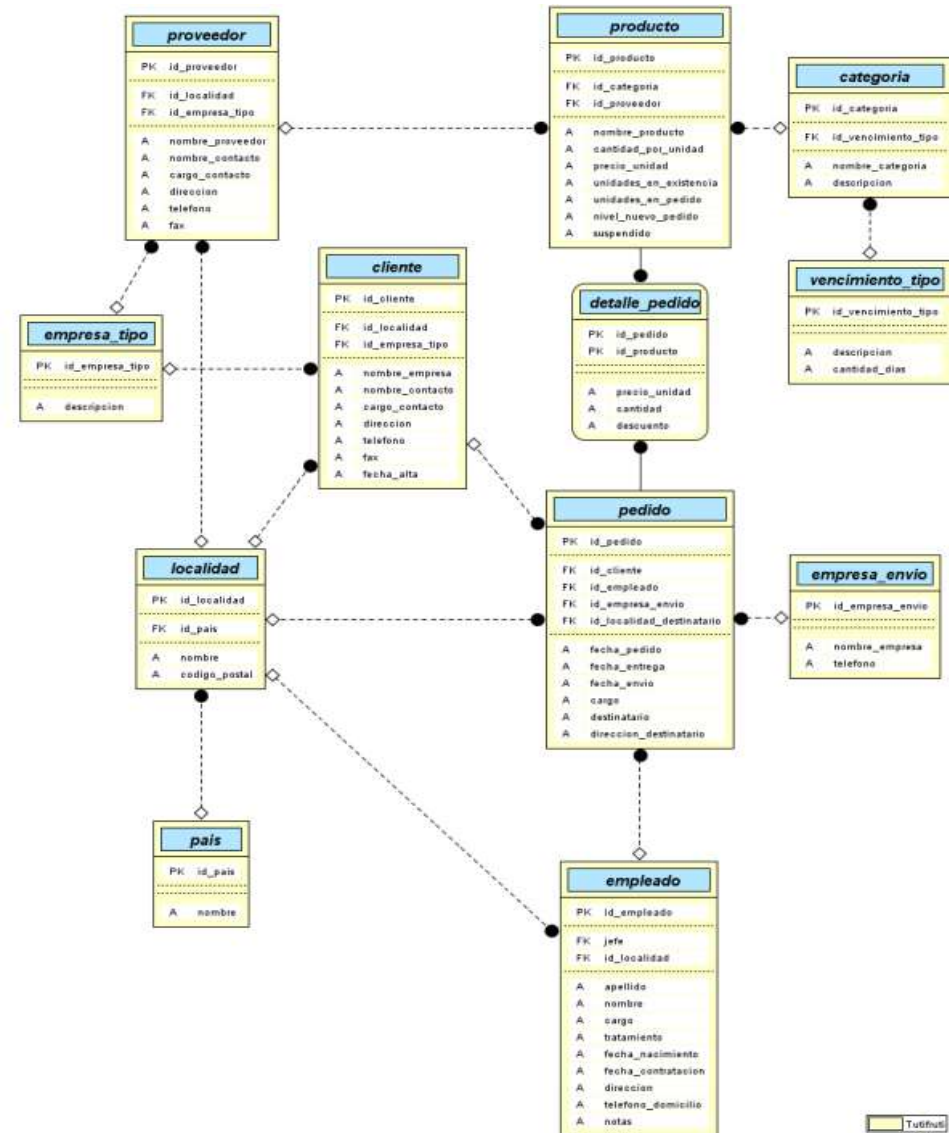


## Encontrar el Mayor/Menor de todos en un conjunto

“Encontrar la categoría de productos, que posee la mayor cantidad de productos”.

```

Select c.id_categoria
From categoría c join producto p on (c.id_categoria = p.id_categoria)
Group by c.id_categoria
Having count(*) >= ALL (Select c1.id_categoria
                        From categoría c1 join producto p1 on
                          (c1.id_categoria = p1.id_categoria)
                        Group by c1.id_categoria)
    
```



Encontrar el Mayor/Menor de todos en un conjunto

“Encontrar la categoría de productos, que posee la mayor cantidad de productos”.

Query Editor   Query History

```
1 Select c.id_categoria
2 from main.categoria c join main.producto p on (c.id_categoria = p.id_categoria)
3 group by c.id_categoria
4 having count(*) >=ALL (select count(*)
5                        from main.categoria c1 join main.producto p1 on (c1.id_categoria = p1.id_categoria)
6                        group by c1.id_categoria);
7
```

|   | Data Output                  | Explain         | Message: |
|---|------------------------------|-----------------|----------|
|   | id_categoria<br>[PK] integer | count<br>bigint |          |
| 1 |                              | 8               | 12       |
| 2 |                              | 7               | 5        |
| 3 |                              | 1               | 12       |
| 4 |                              | 5               | 7        |
| 5 |                              | 4               | 10       |
| 6 |                              | 2               | 12       |
| 7 |                              | 6               | 6        |
| 8 |                              | 3               | 13       |

La consulta externa muestra aquellas categorías cuya cantidad de productos sea mayor o igual a todas las cantidades encontradas en la subconsulta.

|   | Data Output     | Explain | Messages | Notifications |
|---|-----------------|---------|----------|---------------|
|   | count<br>bigint |         |          |               |
| 1 | 12              |         |          |               |
| 2 | 5               |         |          |               |
| 3 | 12              |         |          |               |
| 4 | 7               |         |          |               |
| 5 | 10              |         |          |               |
| 6 | 12              |         |          |               |
| 7 | 6               |         |          |               |
| 8 | 13              |         |          |               |

La subconsulta muestra, la cantidad de productos por categoría.

Encontrar el Mayor/Menor de todos en un conjunto

“Encontrar la categoría de productos, que posee la mayor cantidad de productos”.

Query Editor   Query History

```
1 Select c.id_categoria
2 from main.categoria c join main.producto p on (c.id_categoria = p.id_categoria)
3 group by c.id_categoria
4 having count(*) >=ALL (select count(*)
5                        from main.categoria c1 join main.producto p1 on (c1.id_categoria = p1.id_categoria)
6                        group by c1.id_categoria);
7
```

| Data Output        |                              |                 |    |
|--------------------|------------------------------|-----------------|----|
| Explain   Messages |                              |                 |    |
|                    | id_categoria<br>[PK] integer | count<br>bigint |    |
| 1                  |                              | 8               | 12 |
| 2                  |                              | 7               | 5  |
| 3                  |                              | 1               | 12 |
| 4                  |                              | 5               | 7  |

La consulta externa muestra aquellas categorías cuya cantidad de productos sea mayor o igual a todas las cantidades encontradas en la subconsulta. Para ello, a cada tupla del FROM la compara con cada una de las tuplas de la subconsulta.

| Data Output                        |                 |    |  |
|------------------------------------|-----------------|----|--|
| Explain   Messages   Notifications |                 |    |  |
|                                    | count<br>bigint |    |  |
| 1                                  | OK              | 12 |  |
| 2                                  | OK              | 5  |  |
| 3                                  | OK              | 12 |  |
| 4                                  | OK              | 7  |  |
| 5                                  | OK              | 10 |  |
| 6                                  | OK              | 12 |  |
| 7                                  |                 | 6  |  |
| 8                                  | X               | 13 |  |

La subconsulta muestra, la cantidad de productos por categoría.

12 no es mayor o igual a todos los valores de la subconsulta, por lo tanto id\_categoria 8 no va a la respuesta. Repetí el análisis para los restantes valores....

Encontrar el Mayor/Menor de todos en un conjunto

“Encontrar la categoría de productos, que posee la mayor cantidad de productos”.

Query EditorQuery History

```
1 Select c.id_categoria
2 from main.categoria c join main.producto p on (c.id_categoria = p.id_categoria)
3 group by c.id_categoria
4 having count(*) >=ALL (select count(*)
5                        from main.categoria c1 join main.producto p1 on (c1.id_categoria = p1.id_categoria)
6                        group by c1.id_categoria);
7
```

|   | [PK] integer | bigint |
|---|--------------|--------|
| 1 | 8            | 12     |
| 2 | 7            | 5      |
| 3 | 1            | 12     |
| 4 | 5            | 7      |
| 5 | 4            | 10     |
| 6 | 2            | 12     |
| 7 | 6            | 6      |
| 8 | 3            | 13     |

Data OutputExplainMessagesNotifications

|   | count | bigint |
|---|-------|--------|
| 1 | OK    | 12     |
| 2 | OK    | 5      |
| 3 | OK    | 12     |
| 4 | OK    | 7      |
| 5 | OK    | 10     |
| 6 | OK    | 12     |
| 7 | OK    | 6      |
| 8 | ok    | 13     |

13 mayor o igual a todos los valores de la subconsulta, por lo tanto id\_categoria 3 va a la respuesta.

Encontrar el Mayor/Menor de todos en un conjunto

“Encontrar la categoría de productos, que posee la mayor cantidad de productos”.

Query Editor   Query History

```
1 Select c.id_categoria
2 from main.categoria c join main.producto p on (c.id_categoria = p.id_categoria)
3 group by c.id_categoria
4 having count(*) >=ALL (select count(*)
5                        from main.categoria cl join main.producto pl on (cl.id_categoria = pl.id_categoria)
6                        group by cl.id_categoria);
7
```

| Data Output |                              |   |                 | Explain | Messages | Notifications                                                                |
|-------------|------------------------------|---|-----------------|---------|----------|------------------------------------------------------------------------------|
|             | id_categoria<br>[PK] integer |   | count<br>bigint |         |          |                                                                              |
| 1           |                              | 3 | 13              |         |          | La categoría id_categoria 3, es la que tiene la mayor cantidad de productos. |
|             |                              |   |                 |         |          |                                                                              |

*Podrían existir más de una tupla con mayor/menor cantidad y de esta manera se estarían mostrando todas ellas correctamente.*

## Consultas con “solo” de un conjunto

“Encontrar los proveedores que **sólo** proveen productos de la categoría 5 (y de ninguna otra)”.

### Primera estrategia: NOT EXISTS

La respuesta esperada son los proveedores para los cuales no existen productos de otra categoría que sea provisto por él.

```
SELECT DISTINCT p1.id_proveedor
FROM main.producto p1
WHERE p1.id_categoria = 5
  AND NOT EXISTS (
    SELECT *
    FROM main.producto p2
    WHERE p2.id_proveedor = p1.id_proveedor AND p2.id_categoria <> 5
  );
```



Consultas con “solo” de un conjunto

“Encontrar los proveedores que *sólo* proveen productos de la categoría 5 (y de ninguna otra)”.

Primera estrategia: NOT EXISTS

La respuesta esperada son los proveedores para los cuales no existen productos de otra categoría que sea provisto por él.

Query Editor   Query History

```
12
13
14 SELECT DISTINCT p1.id_proveedor
15 FROM main.producto p1
16 WHERE p1.id_categoria = 5
17    AND NOT EXISTS (
18         SELECT *
19         FROM main.producto p2
20         WHERE p2.id_proveedor = p1.id_proveedor AND p2.id_categoria <> 5
21    );
```

Data Output   Explain   Message

|   | id_proveedor |  |
|---|--------------|--|
|   | integer      |  |
| 1 | 24           |  |
| 2 | 9            |  |
| 3 | 26           |  |
| 4 | 12           |  |
| 5 | 20           |  |

La consulta externa, toma cada tupla de la tabla producto donde id\_categoria es 5 y con el p1.id\_proveedor de esa tupla arma la subconsulta.

Consultas con “solo” de un conjunto

“Encontrar los proveedores que **sólo** proveen productos de la categoría 5 (y de ninguna otra)”.

Primera estrategia: NOT EXISTS

La respuesta esperada son los proveedores para los cuales no existen productos de otra categoría que sea provisto por él.

Query Editor   Query History

```
12
13
14 SELECT DISTINCT p1.id_proveedor
15 FROM main.producto p1
16 WHERE p1.id_categoria = 5
17    AND NOT EXISTS (
18         SELECT *
19         FROM main.producto p2
20         WHERE p2.id_proveedor =
21              );
```

Data Output   Explain   Message

|   | id_proveedor |  |
|---|--------------|--|
|   | integer      |  |
| 1 | 24           |  |
| 2 | 9            |  |
| 3 | 26           |  |
| 4 | 12           |  |
| 5 | 20           |  |

La consulta externa, toma cada tupla de la tabla producto donde id\_categoria es 5 y con el p1.id\_proveedor de esa tupla arma la subconsulta.

La subconsulta correlacionada, muestra todos los productos de categoría distinta que provee el proveedor (p1.id\_proveedor), ej.24

Consultas con “solo” de un conjunto

“Encontrar los proveedores que *sólo* proveen productos de la categoría 5”.

Primera estrategia: NOT EXISTS

La respuesta esperada son los proveedores para los cuales no existen productos de otra categoría que sea provisto por él.

```
16
17 SELECT DISTINCT p1.id_proveedor
18 FROM main.producto p1
19 WHERE p1.id_categoria = 5
20 AND NOT EXISTS (
21     SELECT p2.id_producto, p2.nombre_producto, p2.id_categoria, p2.id_proveedor
22     FROM main.producto p2
23     WHERE p2.id_proveedor = 24 AND p2.id_categoria <> 5
24 );
25
26
```

| Data Output |                          |                                        |                      |                      | Explain | Messages | Notifications |
|-------------|--------------------------|----------------------------------------|----------------------|----------------------|---------|----------|---------------|
|             | id_producto [PK] integer | nombre_producto character varying (40) | id_categoria integer | id_proveedor integer |         |          |               |
| 1           | 53                       | Empanada de carne                      | 6                    | 24                   |         |          |               |
| 2           | 51                       | Manzanas secas Manjimup                | 7                    | 24                   |         |          |               |

| Data Output |                      | Explain | Message |
|-------------|----------------------|---------|---------|
|             | id_proveedor integer |         |         |
| 1           | 24                   |         |         |
| 2           | 9                    |         |         |
| 3           | 26                   |         |         |
| 4           | 12                   |         |         |
| 5           | 20                   |         |         |

La subconsulta correlacionada, muestra todos los productos de categoría distinta que provee el proveedor (p1.id\_proveedor), ej.24

Es el conjunto de todos los proveedores que proveen productos de categoría 5.

Consultas con “solo” de un conjunto

“Encontrar los proveedores que *sólo* proveen productos de la categoría 5”.

Primera estrategia: NOT EXISTS

La respuesta esperada son los proveedores para los cuales no existen productos de otra categoría que sea provisto por él.

```
16
17 SELECT DISTINCT p1.id_proveedor
18 FROM main.producto p1
19 WHERE p1.id_categoria = 5
20 AND NOT EXISTS (
21     SELECT p2.id_producto, p2.nombre_producto, p2.id_categoria, p2.id_proveedor
22     FROM main.producto p2
23     WHERE p2.id_proveedor = 24 AND p2.id_categoria <> 5
24 );
25
26
```

| Data Output |                             |                                           |                         |                         | Explain | Messages | Notifications |
|-------------|-----------------------------|-------------------------------------------|-------------------------|-------------------------|---------|----------|---------------|
|             | id_producto<br>[PK] integer | nombre_producto<br>character varying (40) | id_categoria<br>integer | id_proveedor<br>integer |         |          |               |
| 1           | 53                          | Empanada de carne                         | 6                       | 24                      |         |          |               |
| 2           | 51                          | Manzanas secas Manjimup                   | 7                       | 24                      |         |          |               |
|             |                             |                                           |                         |                         |         |          |               |

Como en este caso la subconsulta correlacionada ha devuelto tuplas, el NOT EXISTS, será FALSO, por lo tanto el proveedor con id 24, no estará en respuesta de la consulta externa, dado que también provee productos de categoría distinta a 5. Repetir este análisis para todos los proveedores que proveen productos categoría 5.

Consultas con “solo” de un conjunto

“Encontrar los proveedores que *sólo* proveen productos de la categoría 5”.

Primera estrategia: NOT EXISTS

La respuesta esperada son los proveedores para los cuales no existen productos de otra categoría que sea provisto por él.

```
10
17 SELECT DISTINCT p1.id_proveedor
18 FROM main.producto p1
19 WHERE p1.id_categoria = 5
20 AND NOT EXISTS (
21     SELECT p2.id_producto, p2.nombre_producto, p2.id_categoria, p2.id_proveedor
22     FROM main.producto p2
23     WHERE p2.id_proveedor = 9 AND p2.id_categoria <> 5
24 );
25
26
```

| Data Output |                             |                                           |                         |                         | Explain | Messages | Notifications |
|-------------|-----------------------------|-------------------------------------------|-------------------------|-------------------------|---------|----------|---------------|
|             | id_producto<br>[PK] integer | nombre_producto<br>character varying (40) | id_categoria<br>integer | id_proveedor<br>integer |         |          |               |
|             |                             |                                           |                         |                         |         |          |               |
|             |                             |                                           |                         |                         |         |          |               |

Eneste caso la subconsulta correlacionada NO devuelve tuplas, el NOT EXISTS, será VERDADERO, por lo tanto el proveedor con id 9, estará en respuesta de la consulta externa, dado que NO provee productos de categoría distinta a 5.

Consultas con “solo” de un conjunto

“Encontrar los proveedores que *sólo* proveen productos de la categoría 5 (y de ninguna otra)”.

Primera estrategia: NOT EXISTS

La respuesta esperada son los proveedores para los cuales no existen productos de otra categoría que sea provisto por él.

```
16
17 SELECT DISTINCT p1.id_proveedor
18 FROM main.producto p1
19 WHERE p1.id_categoria = 5
20    AND NOT EXISTS (
21         SELECT p2.id_producto, p2.nombre_producto, p2.id_categoria, p2.id_proveedor
22         FROM main.producto p2
23         WHERE p2.id_proveedor = p1.id_proveedor AND p2.id_categoria <> 5
24     );
25
26
```

Data Output   Explain   Messages   Notifications

|   | id_proveedor |  |
|---|--------------|--|
|   | integer      |  |
| 1 | 9            |  |
| 2 | 26           |  |

Los proveedores con id 9 y 26, proveen productos que SOLO son de categoría 5 y de ninguna otra.

## Consultas con “solo” de un conjunto

“Encontrar los proveedores que **sólo** proveen productos de la categoría 5”.

### Segunda estrategia: EXCEPT

Los proveedores de productos categoría 5 menos los proveedores productos distintos a la categoría 5

```
(SELECT DISTINCT p1.id_proveedor  
  FROM main.producto p1  
  WHERE p1.id_categoria = 5)
```

EXCEPT

```
(SELECT DISTINCT p2.id_proveedor  
  FROM main.producto p2  
  WHERE p2.id_categoria <>5);
```

Conjunto de los proveedores de  
productos categoría 5

Conjunto de los proveedores de  
productos distintos de categoría 5

Consultas con “solo” de un conjunto

“Encontrar los proveedores que *sólo* proveen productos de la categoría 5”.

Segunda estrategia: EXCEPT

Los proveedores de productos categoría 5 menos los proveedores productos distintos a la categoría 5

28

29 --Estrategia con EXCEPT

30 (SELECT DISTINCT p1.id\_proveedor

31 FROM main.producto p1

32 WHERE p1.id\_categoria = 5)

33 EXCEPT

34 (SELECT DISTINCT p2.id\_proveedor

35 FROM main.producto p2

36 WHERE p2.id\_categoria <>5);

37

38

Data Output

Explain

Messages

Notifications

|   | id_proveedor |  |
|---|--------------|--|
|   | integer      |  |
| 1 | 24           |  |
| 2 | 9            |  |
| 3 | 26           |  |
| 4 | 12           |  |

Conjunto de los proveedores de productos categoría 5

28

29 --Estrategia con EXCEPT

30 (SELECT DISTINCT p1.id\_proveedor

31 FROM main.producto p1

32 WHERE p1.id\_categoria = 5)

33 EXCEPT

34 (SELECT DISTINCT p2.id\_proveedor

35 FROM main.producto p2

36 WHERE p2.id\_categoria <>5);

Data Output

Explain

Messages

Notifications

|    | id_proveedor |  |
|----|--------------|--|
|    | integer      |  |
| 1  | 8            |  |
| 2  | 11           |  |
| 3  | [null]       |  |
| 4  | 29           |  |
| 5  | 4            |  |
| 6  | 10           |  |
| 7  | 7            |  |
| 8  | 15           |  |
| 9  | 6            |  |
| 10 | 12           |  |
| 11 | 24           |  |

Conjunto de los proveedores de productos categoría distinta de 5

Consultas con “solo” de un conjunto

“Encontrar los proveedores que *sólo* proveen productos de la categoría 5”.

Segunda estrategia: EXCEPT

Los proveedores de productos categoría 5 menos los proveedores productos distintos a la categoría 5

29 --Estrategia con EXCEPT  
30 (SELECT DISTINCT p1.id\_proveedor  
31 FROM main.producto p1  
32 WHERE p1.id\_categoria = 5)  
33 EXCEPT  
34 (SELECT DISTINCT p2.id\_proveedor  
35 FROM main.producto p2  
36 WHERE p2.id\_categoria <>5);  
37  
38  
39

Data Output   Explain   Messages   Notifications

|   | id_proveedor<br>integer |  |
|---|-------------------------|--|
| 1 | 9                       |  |
| 2 | 26                      |  |

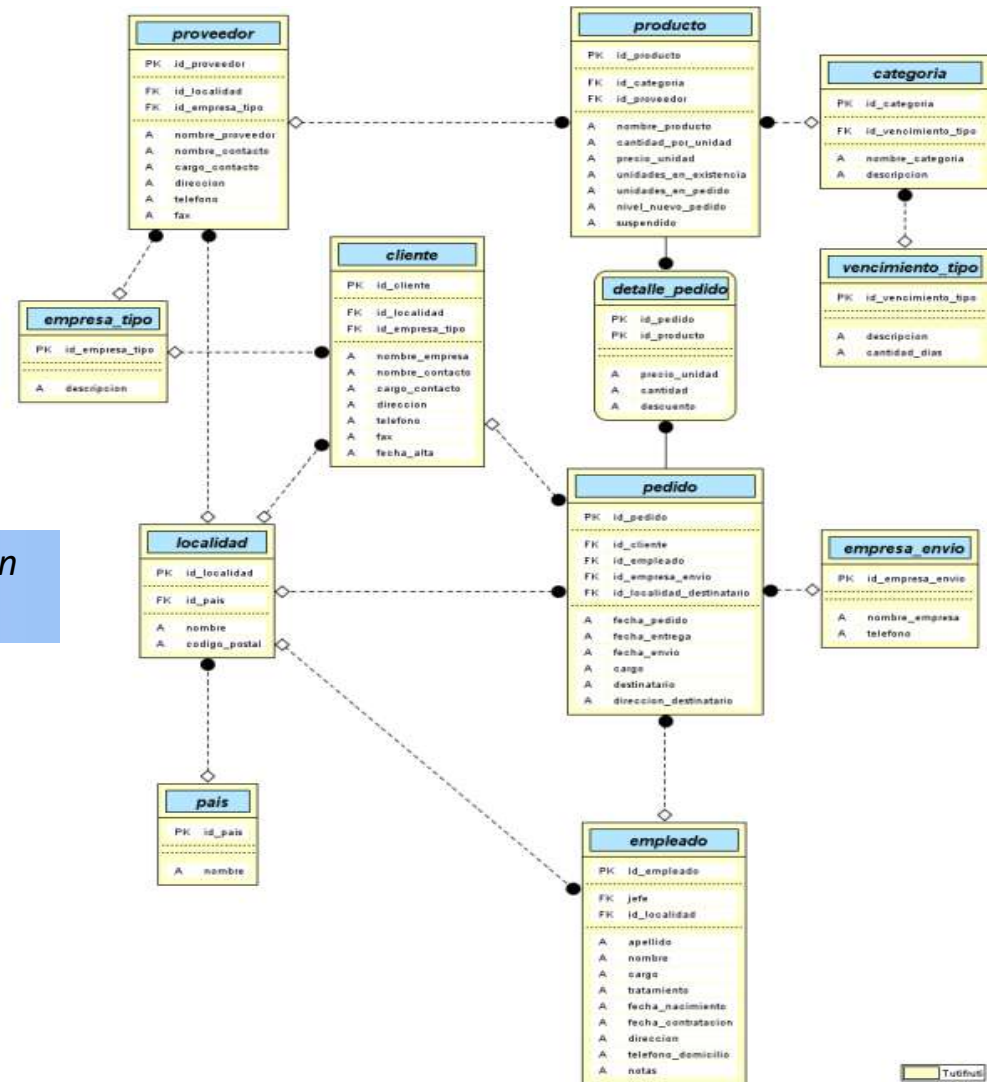
## Consultas “para todos”

Consultas típicas para responder aquellas que incluyen frases como “para todo” ,”al menos uno de cada tipo”

Mostrar los proveedores que tienen productos de todos los tipos de vencimientos



La respuesta son solo los nombres de los proveedores que proveen productos de *TODOS* los tipos de vencimientos posibles.



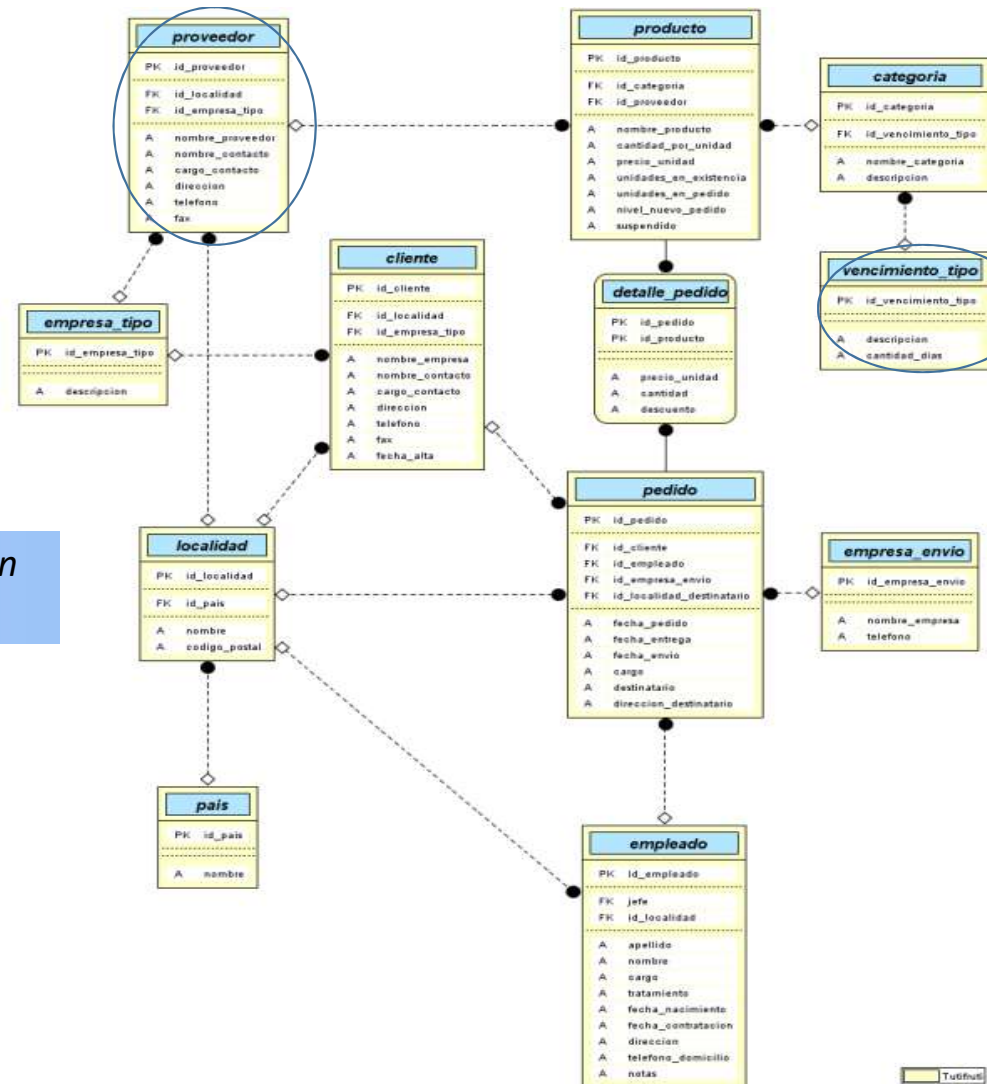
## División del Álgebra Relacional en SQL

Consultas típicas para responder aquellas que incluyen frases como “para todo”, “al menos uno de cada tipo”

Mostrar los proveedores que tienen productos de todos los tipos de vencimientos



La respuesta son solo los nombres de los proveedores que proveen productos de *TODOS* los tipos de vencimientos posibles.



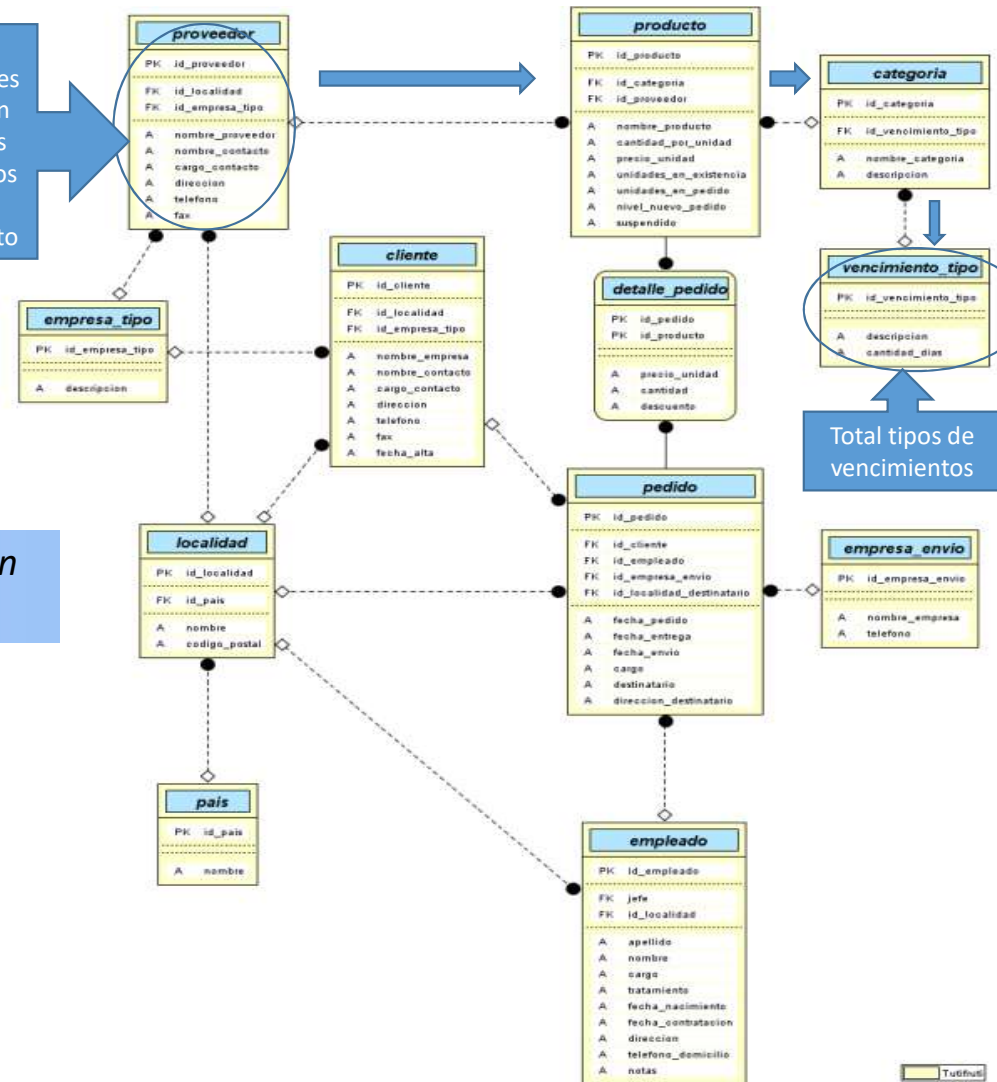
## División del Álgebra Relacional en SQL

Consultas típicas para responder aquellas que incluyen frases como “para todo”, “al menos uno de cada tipo”

Mostrar los proveedores que tienen productos de todos los tipos de vencimientos

La respuesta son solo los nombres de los proveedores que proveen productos de *TODOS* los tipos de vencimientos posibles.

Mostrar proveedores que tienen productos de todos los tipos de vencimiento

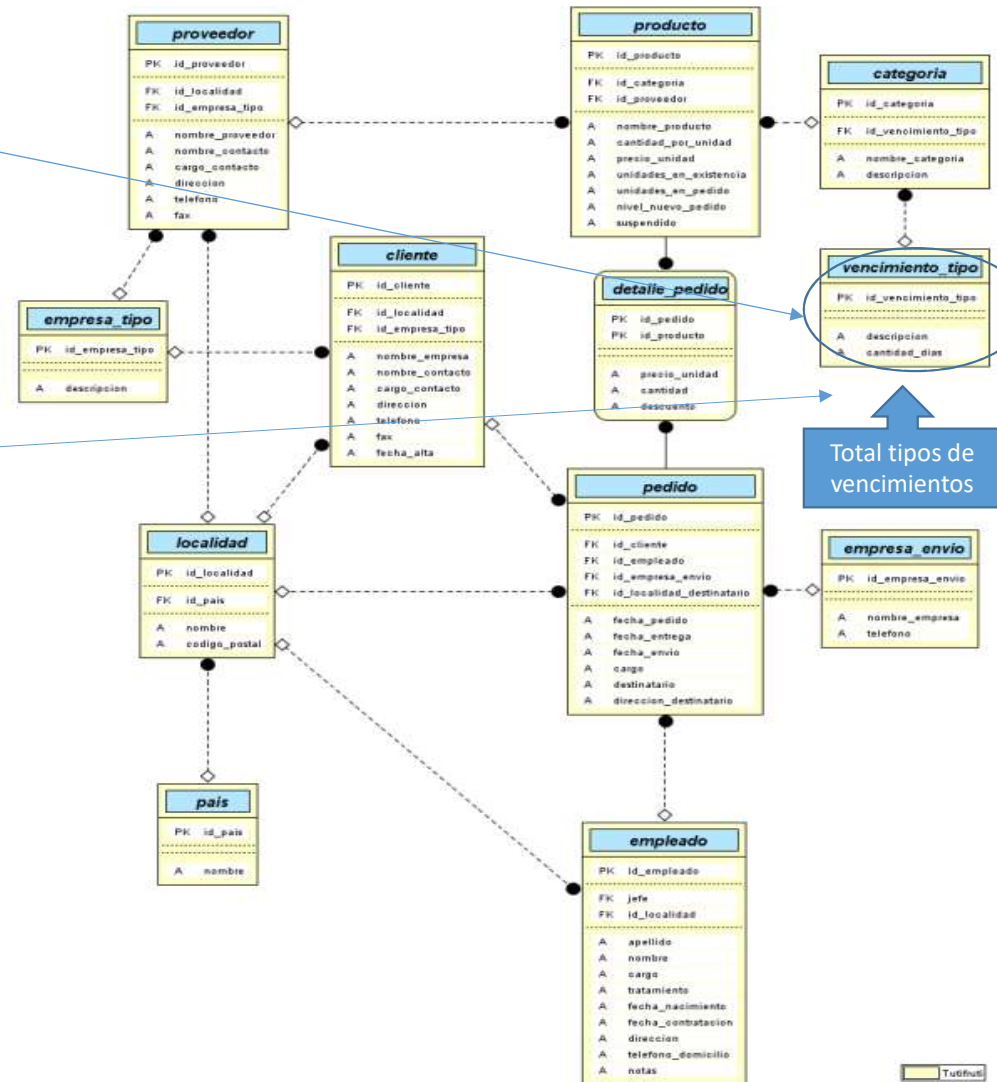


Consultas “para todos”:  
SUBCONSULTAS - AGRUPAMIENTO - COUNT

Mostrar los proveedores que proveen productos de todos  
los tipos de vencimientos

Contar la cantidad total de tipos de vencimientos

Select **count**(id\_vencimiento\_tipo)  
From main.vencimiento\_tipo



Consulta "Para todo":  
SUBCONSULTAS - AGRUPAMIENTO - COUNT

Mostrar los proveedores que proveen productos de todos  
los tipos de vencimientos

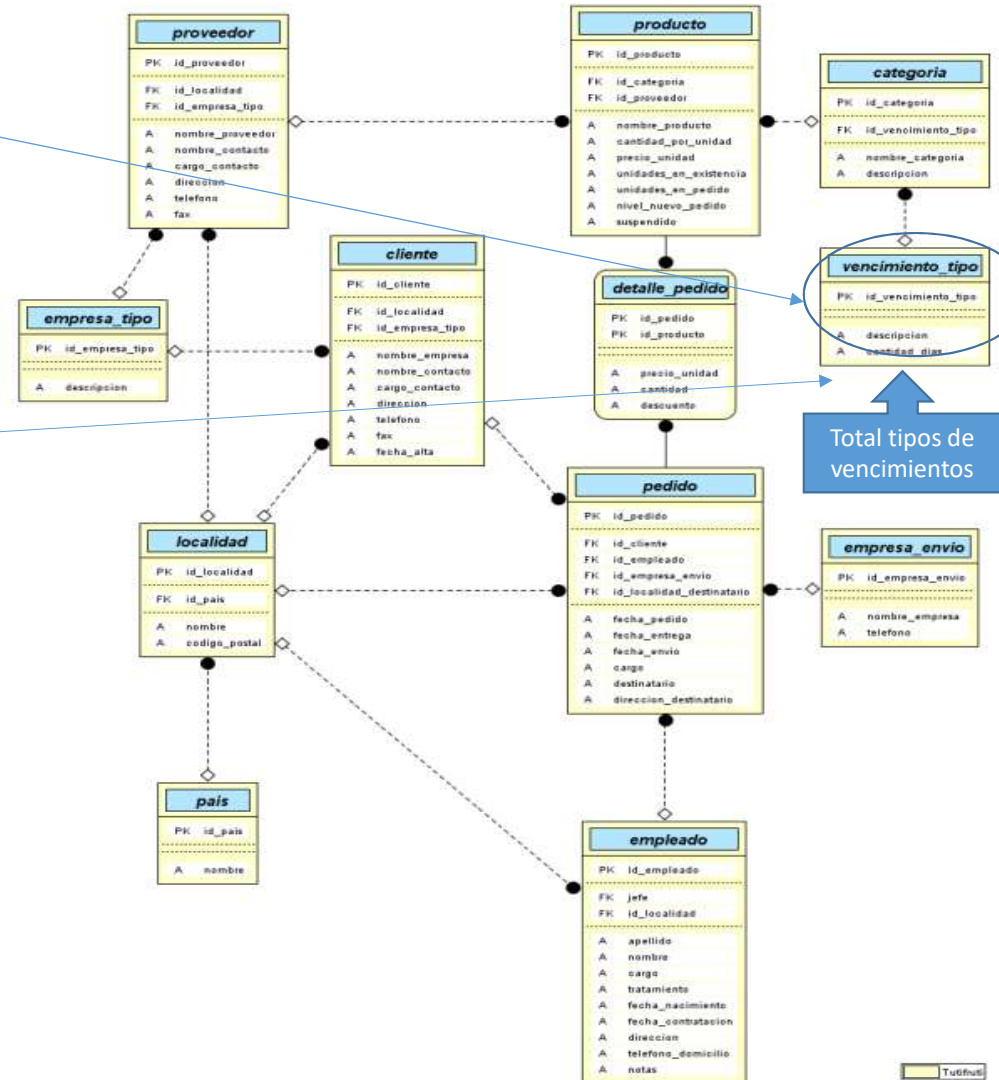
Contar la cantidad total de tipos de vencimientos

Select count(id\_vencimiento\_tipo)  
From main.vencimiento\_tipo



|                                            |                                    |
|--------------------------------------------|------------------------------------|
| 7                                          | select count (id_vencimiento_tipo) |
| 8                                          | from main.vencimiento_tipo)        |
| 9                                          |                                    |
| Data Output Explain Messages Notifications |                                    |
|                                            | count<br>bigint                    |
| 1                                          | 4                                  |

Cantidad Total  
de tipos de  
vencimientos



Consulta “para todos”:  
SUBCONSULTAS - AGRUPAMIENTO - COUNT

Mostrar los proveedores que proveen productos de todos los tipos de vencimientos

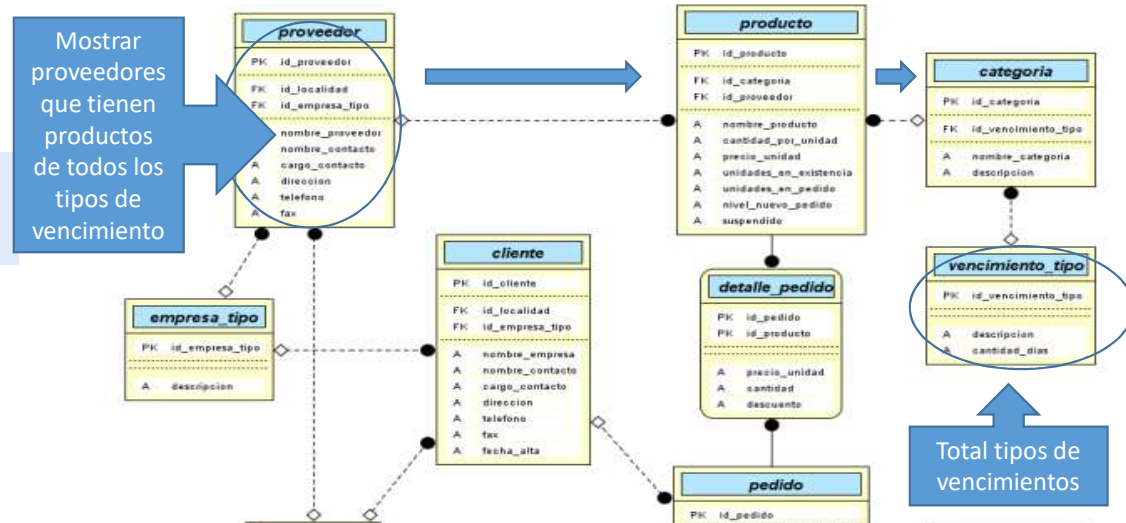
Contar la cantidad total de tipos de vencimientos

```
Select count(id_vencimiento_tipo)
From main.vencimiento_tipo
```

Conocer **cuantos tipos distintos de vencimientos**, tienen los productos que proveen **cada proveedor**



```
select pr.id_proveedor, count(distinct c.id_vencimiento_tipo)
from main.proveedor pr join main.producto
on(pr.id_proveedor = p.id_proveedor)
join main.categoria c on (p.id_categoria = c.id_categoria)
group by pr.id_proveedor
```



```
1 select pr.id_proveedor, count(distinct c.id_vencimiento_tipo)
2 from main.proveedor pr join main.producto p on (pr.id_proveedor = p.id_proveedor)
3 join main.categoria c on (p.id_categoria = c.id_categoria)
4 group by pr.id_proveedor
```

|    | id_proveedor<br>integer | count<br>bigint |
|----|-------------------------|-----------------|
| 2  |                         | 2               |
| 3  |                         | 3               |
| 4  |                         | 4               |
| 5  |                         | 5               |
| 6  |                         | 6               |
| 7  |                         | 7               |
| 8  |                         | 8               |
| 9  |                         | 9               |
| 10 |                         | 10              |
| 11 |                         | 11              |
| 12 |                         | 12              |

Cantidad total de distintos tipos de vencimientos de productos por proveedor

Consulta “para todos”:  
SUBCONSULTAS - AGRUPAMIENTO - COUNT

Mostrar los proveedores que proveen productos de todos los tipos de vencimientos

Contar la cantidad total de tipos de vencimientos



```
Select count(id_vencimiento_tipo)
From main.vencimiento_tipo
```

```
7 select count (id_vencimiento_tipo)
8 from main.vencimiento_tipo
9
```

|   | count  |
|---|--------|
|   | bigint |
| 1 | 4      |

Cantidad Total de Vencimientos Tipos

Conocer cuantos tipos distintos de vencimientos, tienen los productos que proveen cada proveedor



```
select pr.id_proveedor, count(distinct c.id_vencimiento_tipo)
from main.proveedor pr join main.producto p
on(pr.id_proveedor = p.id_proveedor)
join main.categoria c on (p.id_categoria = c.id_categoria)
group by pr.id_proveedor
```

```
1 select pr.id_proveedor, count(distinct c.id_vencimiento_tipo)
2 from main.proveedor pr join main.producto p on(pr.id_proveedor = p.id_proveedor)
3 join main.categoria c on (p.id_categoria = c.id_categoria)
4 group by pr.id_proveedor
```

|    | id_proveedor | count  |
|----|--------------|--------|
|    | integer      | bigint |
| 2  | 2            | 1      |
| 3  | 3            | 2      |
| 4  | 4            | 3      |
| 5  | 5            | 1      |
| 6  | 6            | 2      |
| 7  | 7            | 4      |
| 8  | 8            | 1      |
| 9  | 9            | 1      |
| 10 | 10           | 1      |
| 11 | 11           | 1      |
| 12 | 12           | 4      |

Cantidad total de distintos tipos de vencimientos de productos por proveedor

Consulta “para todos”:  
SUBCONSULTAS - AGRUPAMIENTO - COUNT

Mostrar los proveedores que proveen productos de todos los tipos de vencimientos

Contar la cantidad total de tipos de vencimientos



```
Select count(id_vencimiento_tipo)
From main.vencimiento_tipo
```

```
7 select count (id_vencimiento_tipo)
8 from main.vencimiento_tipo)
9
```

|   | count | bigint |
|---|-------|--------|
| 1 | 4     |        |

Cantidad Total de Vencimientos Tipos

Conocer cuantos tipos distintos de vencimientos, tienen los productos que proveen cada proveedor



```
select pr.id_proveedor, count(distinct c.id_vencimiento_tipo)
from main.proveedor pr join main.producto p
on(pr.id_proveedor = p.id_proveedor)
join main.categoria c on (p.id_categoria = c.id_categoria)
group by pr.id_proveedor
```

Los proveedores 7 y 12 tienen productos de todos los tipos de vencimientos

```
1 select pr.id_proveedor, count(distinct c.id_vencimiento_tipo)
2 from main.proveedor pr join main.producto p on (pr.id_proveedor = p.id_proveedor)
3 join main.categoria c on (p.id_categoria = c.id_categoria)
4 group by pr.id_proveedor
```

|    | id_proveedor | integer | count | bigint |
|----|--------------|---------|-------|--------|
| 2  | 2            |         | 2     | 1      |
| 3  | 3            |         | 3     | 2      |
| 4  | 4            |         | 4     | 3      |
| 5  | 5            |         | 5     | 1      |
| 6  | 6            |         | 6     | 2      |
| 7  | 7            |         | 7     | 4      |
| 8  | 8            |         | 8     | 1      |
| 9  | 9            |         | 9     | 1      |
| 10 | 10           |         | 10    | 1      |
| 11 | 11           |         | 11    | 1      |
| 12 | 12           |         | 12    | 4      |

Cantidad total de distintos tipos de vencimientos de productos por proveedor

Consulta “para todos”:  
SUBCONSULTAS - AGRUPAMIENTO - COUNT

Mostrar los proveedores que proveen productos de todos los tipos de vencimientos

Contar la cantidad total de tipos de vencimientos

Select count(id\_vencimiento\_tipo)  
From main.vencimiento\_tipo

```
7 select count (id_vencimiento_tipo)
8 from main.vencimiento_tipo
9
```

| Data Output |                 | Explain | Messages | Notifications |
|-------------|-----------------|---------|----------|---------------|
|             | count<br>bigint |         |          |               |
| 1           | 4               |         |          |               |

Cantidad Total de Vencimientos Tipos

Conocer cuantos tipos distintos de vencimientos, tienen los productos que proveen cada proveedor

select pr.id\_proveedor, count(distinct c.id\_vencimiento\_tipo)  
from main.proveedor pr join main.producto p  
on(pr.id\_proveedor = p.id\_proveedor)  
join main.categoria c on (p.id\_categoria = c.id\_categoria)  
group by pr.id\_proveedor

Los proveedores 7 y 12  
tienen productos de todos  
los tipos de vencimientos

```
1 select pr.id_proveedor, count(distinct c.id_vencimiento_tipo)
2 from main.proveedor pr join main.producto p on (pr.id_proveedor = p.id_proveedor)
3 join main.categoria c on (p.id_categoria = c.id_categoria)
4 group by pr.id_proveedor
```

| Data Output |                         |                 | Explain | Messages | Notifications |
|-------------|-------------------------|-----------------|---------|----------|---------------|
|             | id_proveedor<br>integer | count<br>bigint |         |          |               |
| 2           | 2                       | 1               |         |          |               |
| 3           | 3                       | 2               |         |          |               |
| 4           | 4                       | 3               |         |          |               |
| 5           | 5                       | 1               |         |          |               |
| 6           | 6                       | 2               |         |          |               |
| 7           | 7                       | 4               |         |          |               |
| 8           | 8                       | 1               |         |          |               |
| 9           | 9                       | 1               |         |          |               |
| 10          | 10                      | 1               |         |          |               |
| 11          | 11                      | 1               |         |          |               |
| 12          | 12                      | 4               |         |          |               |

Cantidad total de distintos tipos de vencimientos de productos por proveedor

¿Puedo encontrar esta respuesta unificando estas consultas?

Consulta “para todos”:  
SUBCONSULTAS - AGRUPAMIENTO - COUNT

Mostrar los proveedores que proveen productos de todos  
los tipos de vencimientos

```
select pr.id_proveedor, count(distinct c.id_vencimiento_tipo)
from
    main.proveedor pr join main.producto p on(pr.id_proveedor = p.id_proveedor)
    join main.categoria c on (p.id_categoria = c.id_categoria)
group by pr.id_proveedor
having count(distinct c.id_vencimiento_tipo) =
    (select count (id_vencimiento_tipo)
     from main.vencimiento_tipo)
```

Consulta que muestra los  
proveedores que tienen  
productos de todos los tipos  
de vencimientos

SubConsulta que  
devuelve Cantidad Total  
de Vencimientos Tipos

Condición: grupos de proveedores cuya  
cantidad de tipos distintos de  
vencimientos de productos es igual a la  
cantidad total de tipos de vencimientos

Consulta “para todos”:  
SUBCONSULTAS - AGRUPAMIENTO - COUNT

Mostrar los proveedores que proveen productos de todos los tipos de vencimientos

```
select pr.id_proveedor, count(distinct c.id_vencimiento_tipo)
from
    main.proveedor pr join main.producto p on(pr.id_proveedor = p.id_proveedor)
    join main.categoria c on (p.id_categoria = c.id_categoria)
group by pr.id_proveedor
having count(distinct c.id_vencimiento_tipo) =
```

(select count (id\_vencimiento\_tipo)  
from main.vencimiento\_tipo)

Consulta que muestra los proveedores que tienen productos de todos los tipos de vencimientos

SubConsulta que devuelve Cantidad Total de Vencimientos Tipos

Condición: grupos de proveedores cuya cantidad de tipos distintos de vencimientos de productos es igual a la cantidad total de tipos de vencimientos tipificados en la tabla correspondiente.

```
1 select pr.id_proveedor, count(distinct c.id_vencimiento_tipo)
2 from
3     main.proveedor pr join main.producto p on(pr.id_proveedor = p.id_proveedor)
4     join main.categoria c on (p.id_categoria = c.id_categoria)
5 group by pr.id_proveedor
6 having count(distinct c.id_vencimiento_tipo) =
7     (select count (id_vencimiento_tipo)
8      from main.vencimiento_tipo)
```

Data Output Explain Messages Notifications

|   | id_proveedor<br>integer | count<br>bigint |
|---|-------------------------|-----------------|
| 1 | 7                       | 4               |
| 2 | 12                      | 4               |

Consulta “para todos”:  
SUBCONSULTAS - AGRUPAMIENTO - COUNT

Mostrar los proveedores que proveen productos de todos  
los tipos de vencimientos

```
select pr.id_proveedor, count(distinct c.id_vencimiento_tipo)
from
    main.proveedor pr join main.producto p on(pr.id_proveedor = p.id_proveedor)
    join main.categoria c on (p.id_categoria = c.id_categoria)
group by pr.id_proveedor
having count(distinct c.id_vencimiento_tipo) =
    (select count (id_vencimiento_tipo)
     from main.vencimiento_tipo)
```

```
1 select pr.id_proveedor, count(distinct c.id_vencimiento_tipo)
2 from main.proveedor pr join main.producto p on(pr.id_proveedor = p.id_proveedor)
3 join main.categoria c on (p.id_categoria = c.id_categoria)
4 group by pr.id_proveedor
```

| Data Output  |        |   | Explain | Messages | Notifications |
|--------------|--------|---|---------|----------|---------------|
| id_proveedor | count  |   |         |          |               |
| integer      | bigint |   |         |          |               |
| 2            | 2      | 1 |         |          |               |
| 3            | 3      | 2 |         |          |               |
| 4            | 4      | 3 |         |          |               |
| 5            | 5      | 1 |         |          |               |
| 6            | 6      | 2 |         |          |               |
| 7            | 7      | 4 |         |          |               |
| 8            | 8      | 1 |         |          |               |
| 9            | 9      | 1 |         |          |               |
| 10           | 10     | 1 |         |          |               |
| 11           | 11     | 1 |         |          |               |
| 12           | 12     | 4 |         |          |               |

```
7 select count (id_vencimiento_tipo)
8 from main.vencimiento_tipo
9
```

| Data Output |   |  | Explain | Messages | Notifications |
|-------------|---|--|---------|----------|---------------|
| count       |   |  |         |          |               |
| bigint      |   |  |         |          |               |
| 1           | 4 |  |         |          |               |

Aplicando la cláusula HAVING

```
1 select pr.id_proveedor, count(distinct c.id_vencimiento_tipo)
2 from
3     main.proveedor pr join main.producto p on(pr.id_proveedor = p.id_proveedor)
4     join main.categoria c on (p.id_categoria = c.id_categoria)
5 group by pr.id_proveedor
6 having count(distinct c.id_vencimiento_tipo) =
7     (select count (id_vencimiento_tipo)
8      from main.vencimiento_tipo)
```

| Data Output  |        |   | Explain | Messages | Notifications |
|--------------|--------|---|---------|----------|---------------|
| id_proveedor | count  |   |         |          |               |
| integer      | bigint |   |         |          |               |
| 1            | 7      | 4 |         |          |               |
| 2            | 12     | 4 |         |          |               |

Proveedores que tienen  
productos de todos los  
tipos de vencimientos

# SQL

## Equijoin

Un **equijoin** es una **operación binaria**, combina filas de dos o más tablas pero que selecciona las filas que tienen valores iguales para ciertas columnas (columnas que toman valores del mismo dominio en ambas tablas, por lo general sus claves primarias-foráneas)

```
SELECT    tabla1.columna, tabla2.columna, ...  
FROM      tabla1, tabla2  
WHERE     tabla1.columna1 = tabla2.columna2;
```

### Aspectos a tener en cuenta

- Es conveniente **preceder el nombre de la columna con el nombre de la tabla** por claridad y para mejorar el acceso a la base de datos.
- Si el **mismo nombre de columna aparece en más de una tabla**, entonces se debe **prefijar** con el nombre de la tabla correspondiente.
- Para combinar tablas, se necesita como mínimo una cantidad de **condiciones de igualdad equivalentes a la cantidad de tablas menos uno**.  
Esta regla **no aplica si la tabla tiene una clave primaria compuesta**.

# SQL

## Left, Right, Full OUTER JOIN

Un outer join extiende el resultado de un join común (INNER JOIN).

Un outer join retorna todas las filas que satisfacen la condición de join más todas las filas de una tabla que no satisfacen la condición de join

Supongamos un join entre las tablas A y B.

Para escribir una consulta en donde queremos además de la condición de join, todas las filas de A que no satisfacen la condición de join empleamos la notación ANSI: LEFT OUTER JOIN.

Para ver todas las filas de B aunque no satisfacen la condición de join empleamos la notación ANSI: RIGHT OUTER JOIN.

Si queremos ver las filas de ambas (A y B) empleamos un FULL OUTER JOIN.

# SQL

Left, Right, Full OUTER JOIN

SINTAXIS ANSI

SELECT ...

INNER  
LEFT [OUTER]  
FROM table\_name RIGHT [OUTER] JOIN table\_name  
FULL [OUTER]

ON condición de join

WHERE ...

# SQL

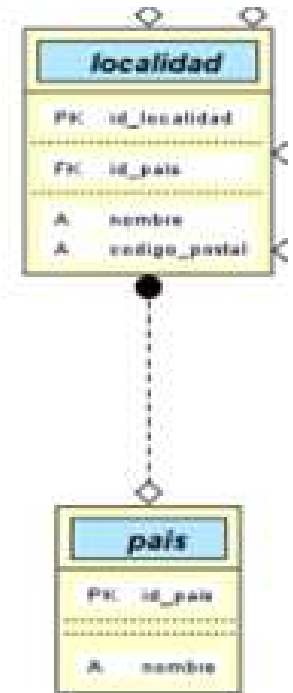
## LEFT OUTER JOIN

Mostrar todos los países y sus localidades, aunque estas últimas no estén registradas.

```
Select p.id_pais, p.nombre as "Pais", l.id_localidad, l.nombre as "Localidad"  
from main.pais p LEFT OUTER JOIN main.localidad l on (p.id_pais = l.id_pais)
```

LEFT OUTER JOIN (o sólo LEFT JOIN), funciona igual que el JOIN, con la salvedad que para todas las tuplas de la relación a la izquierda en el JOIN, en este caso main.país, van al resultado a pesar de no satisfacer la condición del JOIN. Por ello muestra valores para las columnas proyectadas de esta relación main.pais y NULL para cada columna proyectada de la otra relación, en el este caso de main.localidad.

*Coloquialmente: "Quiere ver los datos de todos los países en el sistema. Para aquellos países que la condición del join es verdadera, porque el sistema almacena localidades de dicho país, muestra las tuplas con valores para las columnas proyectadas. Para aquellos países donde la condición del join es falsa, porque el sistema no almacena localidades de dicha país, muestra las tuplas con valores sólo para las columnas proyectadas de país y las restantes con NULL (no existen en la tabla localidad)"*



# SQL

## LEFT OUTER JOIN

Mostrar todos los países y sus localidades, aunque estas últimas no estén registradas.

Query Editor

Query History

```

1 Select p.id_pais, p.nombre as "Pais", l.id_localidad, l.nombre as "Localidad"
2 from main.pais p LEFT OUTER JOIN main.localidad l on (p.id_pais = l.id_pais)

```

Data Output

Explain

Messages

Notifications

|     |  id_pais<br>integer |  Pais<br>character varying (15) |  id_localidad<br>integer |  Localidad<br>character varying (15) |  |
|-----|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 112 | 7                                                                                                  | Canadá                                                                                                           |                                                                                                           | 147                                                                                                                   | Vancouver                                                                         |
| 113 | 13                                                                                                 | Francia                                                                                                          |                                                                                                           | 149                                                                                                                   | Versalles                                                                         |
| 114 | 11                                                                                                 | Estados Unidos                                                                                                   |                                                                                                           | 150                                                                                                                   | Walla Walla                                                                       |
| 115 | 20                                                                                                 | Polonia                                                                                                          |                                                                                                           | 151                                                                                                                   | Warszawa                                                                          |
| 116 | 14                                                                                                 | Holanda                                                                                                          |                                                                                                           | 152                                                                                                                   | Zaandam                                                                           |
| 117 | 11                                                                                                 | Estados Unidos                                                                                                   |                                                                                                           | 47                                                                                                                    | Kirkland                                                                          |
| 118 | 11                                                                                                 | Estados Unidos                                                                                                   |                                                                                                           | 106                                                                                                                   | Redmond                                                                           |
| 119 | 1                                                                                                  | Alemania                                                                                                         |                                                                                                           | 40                                                                                                                    | Frankfurt                                                                         |
| 120 | 25                                                                                                 | Suiza                                                                                                            |                                                                                                           | 41                                                                                                                    | Génova                                                                            |
| 121 | 6                                                                                                  | Brasil                                                                                                           |                                                                                                           | 125                                                                                                                   | Sao Paulo                                                                         |
| 122 | 6                                                                                                  | Brasil                                                                                                           |                                                                                                           | 126                                                                                                                   | Sao Paulo                                                                         |
| 123 | 6                                                                                                  | Brasil                                                                                                           |                                                                                                           | 127                                                                                                                   | Sao Paulo                                                                         |
| 124 | 11                                                                                                 | Estados Unidos                                                                                                   |                                                                                                           | 130                                                                                                                   | Seattle                                                                           |
| 125 | 11                                                                                                 | Estados Unidos                                                                                                   |                                                                                                           | 131                                                                                                                   | Seattle                                                                           |
| 126 | 11                                                                                                 | Estados Unidos                                                                                                   |                                                                                                           | 142                                                                                                                   | Tacoma                                                                            |
| 127 | 6                                                                                                  | Brasil                                                                                                           |                                                                                                           | 128                                                                                                                   | Sao Paulo                                                                         |
| 128 | 6                                                                                                  | Brasil                                                                                                           |                                                                                                           | 129                                                                                                                   | Sao Paulo                                                                         |
| 129 | 9                                                                                                  | EE.UU.                                                                                                           | [null]                                                                                                    | [null]                                                                                                                |                                                                                   |

En este caso el p

registradas en la

En este caso el país EE.UU no tiene localidades registradas en la tabla Localidades

# SQL

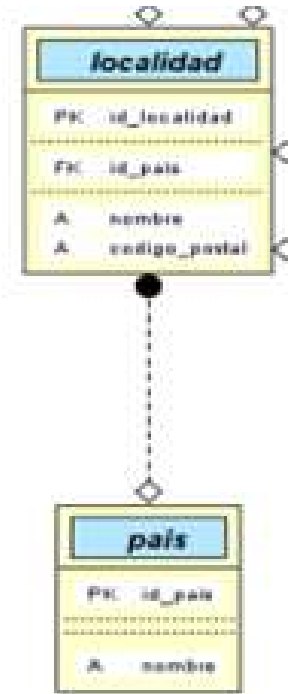
## RIGHT OUTER JOIN

Mostrar los países y todas las localidades, aunque estas últimas no registren país de pertenencia

```
Select p.id_pais, p.nombre as "Pais", l.id_localidad, l.nombre as "Localidad"  
from main.pais p RIGH OUTER JOIN main.localidad l on (p.id_pais = l.id_pais)
```

RIGHT OUTER JOIN (o sólo RIGHT JOIN), funciona igual que el JOIN, con la salvedad que para todas las tuplas de la relación a la derecha en el JOIN, en este caso main.localidad, van al resultado a pesar de no satisfacer la condición del JOIN. Por ello muestra valores para las columnas proyectadas de esta relación main.localidad y NULL para cada columna proyectada de la otra relación, en el este caso de main.pais.

*Coloquialmente: "Quiere ver los datos de todas las localidades en el sistema. Para aquellas localidades que la condición del join es verdadera, porque el sistema almacena el país al que pertenece la localidad, muestra las tuplas con valores para todas las columnas proyectadas. Para aquellas localidades donde la condición del join es falsa, porque el sistema no almacena el país de dicha localidad, muestra las tuplas con valores sólo para las columnas proyectadas de localidad y con NULL para las correspondiente a país, (no existen en la tabla pais)*



# SQL

## RIGHT OUTER JOIN

Mostrar los países y todas las localidades, aunque estas últimas no registren país de pertenencia

```
1 Select p.id_pais, p.nombre as "Pais", l.id_localidad, l.nombre as "Localidad"
2 from main.pais p RIGHT OUTER JOIN main.localidad l on (p.id_pais = l.id_pais)
```

|     | <b>id_pais</b><br>integer | <b>Pais</b><br>character varying (15) | <b>id_localidad</b><br>integer | <b>Localidad</b><br>character varying (15) |
|-----|---------------------------|---------------------------------------|--------------------------------|--------------------------------------------|
| 113 | 13                        | Francia                               | 149                            | Versalles                                  |
| 114 | 11                        | Estados Unidos                        | 150                            | Walla Walla                                |
| 115 | 20                        | Polonia                               | 151                            | Warszawa                                   |
| 116 | 14                        | Holanda                               | 152                            | Zaandam                                    |
| 117 | 11                        | Estados Unidos                        | 47                             | Kirkland                                   |
| 118 | 11                        | Estados Unidos                        | 106                            | Redmond                                    |
| 119 | 1                         | Alemania                              | 40                             | Frankfurt                                  |
| 120 | 25                        | Suiza                                 | 41                             | Génova                                     |
| 121 | 6                         | Brasil                                | 125                            | Sao Paulo                                  |
| 122 | 6                         | Brasil                                | 126                            | Sao Paulo                                  |
| 123 | 6                         | Brasil                                | 127                            | Sao Paulo                                  |
| 124 | 11                        | Estados Unidos                        | 130                            | Seattle                                    |
| 125 | 11                        | Estados Unidos                        | 131                            | Seattle                                    |
| 126 | 11                        | Estados Unidos                        | 142                            | Tacoma                                     |
| 127 | 6                         | Brasil                                | 128                            | Sao Paulo                                  |
| 128 | 6                         | Brasil                                | 129                            | Sao Paulo                                  |
| 129 | [null]                    | [null]                                | 1000                           | Mi Localidad                               |
| 130 | [null]                    | [null]                                | 1001                           | Otra Localidad                             |

En este caso las localidades Mi Localidad y Otra Localidad no tienen un país relacionado, por ello muestra las columnas correspondientes en NULL

# SQL

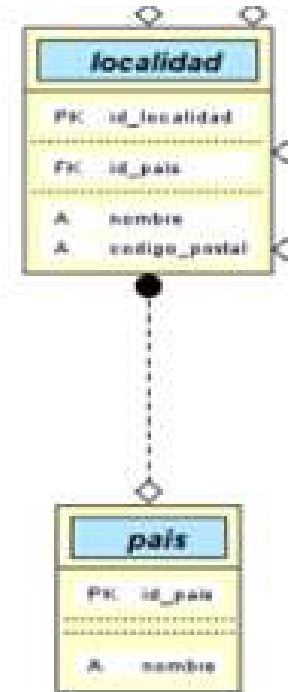
## Full OUTER JOIN

Mostrar todos los países y todas las localidades, aún si no existe relación entre ellas.

```
Select p.id_pais, p.nombre as "Pais", l.id_localidad, l.nombre as "Localidad"  
from main.pais p FULL OUTER JOIN main.localidad l on (p.id_pais = l.id_pais)
```

FULL OUTER JOIN (o sólo FULL JOIN), en el resultado se muestran las tuplas que cumplen la relación del JOIN, y también todas las tuplas que no lo hagan, completando con NULL los valores para las columnas en las que no encuentran coincidencia.

*Coloquialmente: "Quiere ver los datos de todas las localidades y todos los países en el sistema. Para aquella tuplas de pais donde la condición del JOIN es falsa, completa con NULL las columnas proyectas de localidad. Para aquella tuplas de localidad donde la condición del JOIN es falsa, completa con NULL las columnas proyectas de pais."*



# SQL

## Full OUTER JOIN

Mostrar todos los países y todas las localidades, aun si no existe relación entre ellas.

Query Editor

Query History

```
1 Select p.id_pais, p.nombre as "Pais", l.id_localidad, l.nombre as "Localidad"
2 from main.pais p FULL OUTER JOIN main.localidad l on (p.id_pais = l.id_pais)
```

Data Output

Explain

Messages

Notifications

|     | <div>id_pais</div> <div>integer</div> | <div></div> | <div>Pais</div> <div>character varying (15)</div> | <div></div> | <div>id_localidad</div> <div>integer</div> | <div></div> | <div>Localidad</div> <div>character varying (15)</div> | <div></div> |
|-----|---------------------------------------|-------------|---------------------------------------------------|-------------|--------------------------------------------|-------------|--------------------------------------------------------|-------------|
| 114 | 11                                    |             | Estados Unidos                                    |             | 150                                        |             | Walla Walla                                            |             |
| 115 | 20                                    |             | Polonia                                           |             | 151                                        |             | Warszawa                                               |             |
| 116 | 14                                    |             | Holanda                                           |             | 152                                        |             | Zaandam                                                |             |
| 117 | 11                                    |             | Estados Unidos                                    |             | 47                                         |             | Kirkland                                               |             |
| 118 | 11                                    |             | Estados Unidos                                    |             | 106                                        |             | Redmond                                                |             |
| 119 | 1                                     |             | Alemania                                          |             | 40                                         |             | Frankfurt                                              |             |
| 120 | 25                                    |             | Suiza                                             |             | 41                                         |             | Génova                                                 |             |
| 121 | 6                                     |             | Brasil                                            |             | 125                                        |             | Sao Paulo                                              |             |
| 122 | 6                                     |             | Brasil                                            |             | 126                                        |             | Sao Paulo                                              |             |
| 123 | 6                                     |             | Brasil                                            |             | 127                                        |             | Sao Paulo                                              |             |
| 124 | 11                                    |             | Estados Unidos                                    |             | 130                                        |             | Seattle                                                |             |
| 125 | 11                                    |             | Estados Unidos                                    |             | 131                                        |             | Seattle                                                |             |
| 126 | 11                                    |             | Estados Unidos                                    |             | 142                                        |             | Tacoma                                                 |             |
| 127 | 6                                     |             | Brasil                                            |             | 128                                        |             | Sao Paulo                                              |             |
| 128 | 6                                     |             | Brasil                                            |             | 129                                        |             | Sao Paulo                                              |             |
| 129 | [null]                                | [null]      |                                                   |             | 1000                                       |             | Mi Localidad                                           |             |
| 130 | [null]                                | [null]      |                                                   |             | 1001                                       |             | Otra Localidad                                         |             |
| 131 | 9                                     |             | EE.UU.                                            |             | [null]                                     | [null]      |                                                        |             |

**Nota:** Los tres tipos de outer join se ejemplificaron adaptando los datos de las tablas de la BD Tutifruti.

# SQL

## INNER JOIN - JOIN

Mostrar los clientes y los pedidos realizados por éstos

Query Editor Query History

```
1 select p.id_pedido, p.id_cliente, ee.id_empresa_envio, ee.nombre_empresa
2 from main.pedido p join main.empresa_envio ee on (p.id_empresa_envio = ee.id_empresa_envio)
3
```

Data Output Explain Messages Notifications

|    | id_pedido<br>integer | id_cliente<br>character varying (5) | id_empresa_envio<br>integer | nombre_empresa<br>character varying (40) |
|----|----------------------|-------------------------------------|-----------------------------|------------------------------------------|
| 1  | 11069                | TORTU                               |                             | 2 United Package                         |
| 2  | 11068                | QUEEN                               |                             | 2 United Package                         |
| 3  | 11070                | LEHMS                               |                             | 1 Speedy Express                         |
| 4  | 11071                | LILAS                               |                             | 1 Speedy Express                         |
| 5  | 11072                | ERNSH                               |                             | 2 United Package                         |
| 6  | 11073                | PERIC                               |                             | 2 United Package                         |
| 7  | 11074                | SIMOB                               |                             | 2 United Package                         |
| 8  | 11075                | RICSU                               |                             | 2 United Package                         |
| 9  | 11076                | BONAP                               |                             | 2 United Package                         |
| 10 | 11077                | RATTC                               |                             | 2 United Package                         |
| 11 | 11019                | RANCH                               |                             | 3 Federal Shipping                       |
| 12 | 11039                | LINOD                               |                             | 2 United Package                         |
| 13 | 11040                | GREAL                               |                             | 3 Federal Shipping                       |
| 14 | 11045                | BOTTM                               |                             | 2 United Package                         |
| 15 | 11051                | LAMAI                               |                             | 3 Federal Shipping                       |
| 16 | 11054                | CACTU                               |                             | 1 Speedy Express                         |
| 17 | 11058                | BLAUS                               |                             | 3 Federal Shipping                       |

Query Editor Query History

```
1 select p.id_pedido, p.id_cliente, ee.id_empresa_envio, ee.nombre_empresa
2 from main.pedido p INNER join main.empresa_envio ee on (p.id_empresa_envio = ee.id_empresa_envio)
3
```

Data Output Explain Messages Notifications

|    | id_pedido<br>integer | id_cliente<br>character varying (5) | id_empresa_envio<br>integer | nombre_empresa<br>character varying (40) |
|----|----------------------|-------------------------------------|-----------------------------|------------------------------------------|
| 1  | 11069                | TORTU                               |                             | 2 United Package                         |
| 2  | 11068                | QUEEN                               |                             | 2 United Package                         |
| 3  | 11070                | LEHMS                               |                             | 1 Speedy Express                         |
| 4  | 11071                | LILAS                               |                             | 1 Speedy Express                         |
| 5  | 11072                | ERNSH                               |                             | 2 United Package                         |
| 6  | 11073                | PERIC                               |                             | 2 United Package                         |
| 7  | 11074                | SIMOB                               |                             | 2 United Package                         |
| 8  | 11075                | RICSU                               |                             | 2 United Package                         |
| 9  | 11076                | BONAP                               |                             | 2 United Package                         |
| 10 | 11077                | RATTC                               |                             | 2 United Package                         |
| 11 | 11019                | RANCH                               |                             | 3 Federal Shipping                       |
| 12 | 11039                | LINOD                               |                             | 2 United Package                         |
| 13 | 11040                | GREAL                               |                             | 3 Federal Shipping                       |
| 14 | 11045                | BOTTM                               |                             | 2 United Package                         |
| 15 | 11051                | LAMAI                               |                             | 3 Federal Shipping                       |
| 16 | 11054                | CACTU                               |                             | 1 Speedy Express                         |
| 17 | 11058                | BLAUS                               |                             | 3 Federal Shipping                       |

✓ Successfully run. Total query runtime: 211 msec. 830 rows affected.

✓ Successfully run. Total query runtime: 195 msec. 830 rows

INNER JOIN tiene el mismo efecto que JOIN al formar el conjunto de datos en la cláusula FROM. Muestra todos los clientes que tienen pedidos, pero si se necesita incluir en el listado también aquellos clientes que *no tienen actividad*, es decir que no han realizado pedidos.

# SQL

## OUTER JOIN

Listar todos los clientes y los pedidos que hubieran realizado.

```
SELECT c.id_cliente, c.nombre_empresa, p.id_pedido  
FROM main.cliente c left join main.pedido p ON (c.id_cliente = p.id_cliente);
```

Query Editor

Query History

1

2

3

```
SELECT c.id_cliente, c.nombre_empresa, p.id_pedido, p.fecha_pedido
FROM main.cliente c left join main.pedido p ON (c.id_cliente = p.id_cliente)
```

Data Output

Explain

Messages

Notifications

|     | id_cliente            | nombre_empresa                   | id_pedido | fecha_pedido |  |
|-----|-----------------------|----------------------------------|-----------|--------------|--|
|     | character varying (5) | character varying (60)           | integer   | date         |  |
| 815 | WHITC                 | White Clover Markets             | 10596     | 1995-08-11   |  |
| 816 | QUICK                 | QUICK-Stop                       | 10515     | 1995-05-24   |  |
| 817 | SEVES                 | Seven Seas Imports               | 10523     | 1995-06-01   |  |
| 818 | LAZYK                 | Lazy K Kountry Store             | 10545     | 1995-06-22   |  |
| 819 | QUICK                 | QUICK-Stop                       | 10451     | 1995-03-22   |  |
| 820 | WHITC                 | White Clover Markets             | 10483     | 1995-04-24   |  |
| 821 | GOURL                 | Gourmet Lanchonetes              | 10423     | 1995-02-23   |  |
| 822 | PICCO                 | Piccolo und mehr                 | 10427     | 1995-02-27   |  |
| 823 | PRINI                 | Princesa Isabel Vinhos           | 10433     | 1995-03-06   |  |
| 824 | HUNGO                 | Hungry Owl All-Night Grocers     | 10380     | 1995-01-12   |  |
| 825 | BERGS                 | Berglunds snabbköp               | 10280     | 1994-09-14   |  |
| 826 | SUPRD                 | Suprêmes délices                 | 10302     | 1994-10-11   |  |
| 827 | HUNGO                 | Hungry Owl All-Night Grocers     | 10309     | 1994-10-20   |  |
| 828 | WARTH                 | Wartian Herkkku                  | 10320     | 1994-11-03   |  |
| 829 | FOLKO                 | Folk och få HB                   | 10264     | 1994-08-24   |  |
| 830 | SPLIR                 | Split Rail Beer & Ale            | 10271     | 1994-09-01   |  |
| 831 | PARIS                 | Paris spécialités                | [null]    | [null]       |  |
| 832 | FISSA                 | FISSA Fabrica Inter. Salchich... | [null]    | [null]       |  |

Muestra los cli

Muestra los clientes que no han realizado pedidos, que no tienen actividad

# SQL

## OUTER JOIN

Listar **todos** los clientes y sus pedidos a partir del 1/5/1996, si estos existen.

```
SELECT c.id_cliente, c.nombre_empresa, p.id_pedido
FROM main.cliente c left join main.pedido p ON (c.id_cliente = p.id_cliente)
WHERE p.fecha_pedido > '1996-05-01'
```

→ Genera los registros con NULL para los clientes sin pedidos  
→ Filtra las filas que cumplen esta condición, las que tienen NULL no la cumplen y así elimina los clientes sin pedidos.

sin pedidos.

Query Editor

Query History

1

2

3

4

SELECT c.id\_cliente, c.nombre\_empresa, p.id\_pedido, p.fecha\_pedido

FROM main.cliente c left join main.pedido p ON (c.id\_cliente = p.id\_cliente)

WHERE p.fecha\_pedido > '1996-05-01'

Data Output

Explain

Messages

Notifications

id\_cliente

nombre\_empresa

id\_pedido

fecha\_pedido

character varying (5)

character varying (60)

integer

date

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

HANAR

BSBEV

EASTC

WARTH

FRANS

BOTTM

KOENE

CHOPS

SAVEA

SAVEA

WHITC

RICSU

OLDWO

SUPRD

SAVEA

WHITC

DRACD

ERNSH

Hanari Carnes

B's Beverages

Eastern Connection

Wartian Herkku

Franchi S.p.A.

Bottom-Dollar Markets

Königlich Essen

Chop-suey Chinese

Save-a-lot Markets

Save-a-lot Markets

White Clover Markets

Richter Supermarkt

Old World Delicatessen

Suprêmes délices

Save-a-lot Markets

White Clover Markets

Drachenblut Delikatessen

Ernst Handel

11022

11023

11024

11025

11026

11027

11028

11029

11030

11031

11032

11033

11034

11035

11064

11066

11067

11008

1996-05-14

1996-05-14

1996-05-15

1996-05-15

1996-05-15

1996-05-16

1996-05-16

1996-05-16

1996-05-17

1996-05-17

1996-05-17

1996-05-17

1996-05-20

1996-05-20

1996-05-31

1996-05-31

1996-06-03

1996-05-08

Muestra los c

los clientes q

NULL en fech

modificar la c

lo tienen ) des

Muestra los clientes que han realizado pedidos desde 01-05-1996, perdió los clientes que no tienen pedidos. El WHERE eliminó las columnas con NULL en fecha\_pedido. Se comportó igual que un INNER JOIN. ¿Cómo modificar la consultas para que muestre a todos los clientes y sus pedidos (si lo tienen ) desde 01-05-1996?

Así, el WHERE rompe el LEFT JOIN

# SQL

## OUTER JOIN

Listar **todos** los clientes y sus pedidos a partir del 1/5/1996, si éstos existen

```
SELECT c.id_cliente, c.nombre_empresa, p.id_pedido  
FROM main.cliente c left join main.pedido p ON (c.id_cliente = p.id_cliente) and p.fecha_pedido > '1996-05-01';
```

Query Editor

Query History

```
1 SELECT c.id_cliente, c.nombre_empresa, p.id_pedido, p.fecha_pedido
2 FROM main.cliente c left join main.pedido p ON (c.id_cliente = p.id_cliente)
3 and p.fecha_pedido > '1996-05-01'
```

Data Output

Explain

Messages

Notifications

|     | id_cliente<br>character varying (5) | nombre_empresa<br>character varying (60) | id_pedido<br>integer | fecha_pedido<br>date |
|-----|-------------------------------------|------------------------------------------|----------------------|----------------------|
| 82  | WHITC                               | White Clover Markets                     | 11066                | 1996-05-31           |
| 83  | DRACD                               | Drachenblut Delikatessen                 | 11067                | 1996-06-03           |
| 84  | ERNSH                               | Ernst Handel                             | 11008                | 1996-05-08           |
| 85  | SEVES                               | Seven Seas Imports                       | [null]               | [null]               |
| 86  | LAZYK                               | Lazy K Kountry Store                     | [null]               | [null]               |
| 87  | THEBI                               | The Big Cheese                           | [null]               | [null]               |
| 88  | FOLIG                               | Folies gourmandes                        | [null]               | [null]               |
| 89  | CENTC                               | Centro comercial Moctezuma               | [null]               | [null]               |
| 90  | QUEDE                               | Que Delicia                              | [null]               | [null]               |
| 91  | WELLI                               | Wellington Importadora                   | [null]               | [null]               |
| 92  | VICTE                               | Victuailles en stock                     | [null]               | [null]               |
| 93  | GALED                               | Galería del gastrónomo                   | [null]               | [null]               |
| 94  | BOLID                               | Bólido Comidas preparadas                | [null]               | [null]               |
| 95  | ISLAT                               | Island Trading                           | [null]               | [null]               |
| 96  | OCEAN                               | Océano Atlántico Ltda.                   | [null]               | [null]               |
| 97  | LETSS                               | Let's Stop N Shop                        | [null]               | [null]               |
| 98  | MEREP                               | Mère Paillarde                           | [null]               | [null]               |
| 99  | GROSR                               | GROSELLA-Restaurante                     | [null]               | [null]               |
| 100 | BLONP                               | Blondel père et fils                     | [null]               | [null]               |

Muestra todos  
1996. Si tiene  
pedido, de lo  
significa que ta  
pero podrían te

Muestra todos los clientes hayan o no realizado pedidos desde 01-05-1996. Si tienen pedidos en las fechas indicadas coloca los datos del pedido, de lo contrario completa con NULL las columnas proyectadas, significa que tales clientes no han realizado pedidos desde 01-05-1996, pero podrían tener pedidos con fecha anterior.

*Se mantiene el LEFT JOIN real.*

## SQL

### OUTER JOIN

Mostrar los clientes que no han realizado pedidos.

```
SELECT c.id_cliente, c.nombre_empresa, p.id_pedido
FROM main.cliente c left join main.pedido p ON (c.id_cliente = p.id_cliente)
WHERE p.id_pedido IS NULL;
```

Query Editor

Query History

```
1 SELECT c.id_cliente, c.nombre_empresa, p.id_pedido, p.fecha_pedido
2 FROM main.cliente c left join main.pedido p ON (c.id_cliente = p.id_cliente)
3 WHERE p.id_pedido IS NULL;
```

Data Output

Explain

Messages

Notifications

|   | <b>id_cliente</b><br>character varying (5) | <b>nombre_empresa</b><br>character varying (60) | <b>id_pedido</b><br>integer | <b>fecha_pedido</b><br>date |
|---|--------------------------------------------|-------------------------------------------------|-----------------------------|-----------------------------|
| 1 | PARIS                                      | Paris spécialités                               | [null]                      | [null]                      |
| 2 | FISSA                                      | FISSA Fabrica Inter. Salchich...                | [null]                      | [null]                      |

Búsqueda in  
nuestra cli

Búsqueda intencional de clientes inactivos. Sólo muestra clientes sin pedidos.

## SQL

### OUTER JOIN

Mostrar los clientes que no han realizado pedidos.

```
SELECT c.id_cliente, c.nombre_empresa, p.id_pedido
FROM main.cliente c left join main.pedido p ON (c.id_cliente = p.id_cliente)
WHERE p.id_pedido IS NULL;
```

Query Editor

Query History

```
1 SELECT c.id_cliente, c.nombre_empresa, p.id_pedido, p.fecha_pedido
2 FROM main.cliente c left join main.pedido p ON (c.id_cliente = p.id_cliente)
3 WHERE p.id_pedido IS NULL;
```

Data Output

Explain

Messages

Notifications

|   | id_cliente<br>character varying (5) | nombre_empresa<br>character varying (60) | id_pedido<br>integer | fecha_pedido<br>date |
|---|-------------------------------------|------------------------------------------|----------------------|----------------------|
| 1 | PARIS                               | Paris spécialités                        | [null]               | [null]               |
| 2 | FISSA                               | FISSA Fabrica Inter. Salchich...         | [null]               | [null]               |

Búsqueda in  
nuestra cli

Búsqueda intencional de clientes inactivos. Sólo muestra clientes sin pedidos.

# SQL

## OUTER JOIN

Listar todos los clientes junto con la cantidad de pedidos realizados, incluyendo aquellos que no tienen ninguno.

```
SELECT c.id_cliente, c.nombre_empresa, p.id_pedido
FROM main.cliente c left join main.pedido p ON (c.id_cliente = p.id_cliente)
GROUP BY c.nombre_empresa;
```

Query Editor

Query History

```

1 SELECT c.nombre_empresa, COUNT(p.id_pedido) AS cantidad
2 FROM main.cliente c
3 LEFT JOIN main.pedido p
4     ON c.id_cliente = p.id_cliente
5 GROUP BY c.nombre_empresa

```

Data Output

Explain

Messages

Notifications

|    | nombre_empresa<br>character varying (60) | cantidad<br>bigint |
|----|------------------------------------------|--------------------|
| 1  | Paris spécialités                        | 0                  |
| 2  | FISSA Fabrica Inter. Salchich...         | 0                  |
| 3  | Centro comercial Moctezuma               | 1                  |
| 4  | Lazy K Kountry Store                     | 2                  |
| 5  | GROSELLA-Restaurante                     | 2                  |
| 6  | Trail's Head Gourmet Provisi...          | 3                  |
| 7  | Laughing Bacchus Wine Cell...            | 3                  |
| 8  | The Cracker Box                          | 3                  |
| 9  | Consolidated Holdings                    | 3                  |
| 10 | France restauration                      | 3                  |
| 11 | North/South                              | 3                  |
| 12 | Bólido Comidas preparadas                | 3                  |
| 13 | Let's Stop N Shop                        | 4                  |
| 14 | La corne d'abondance                     | 4                  |

Muestra la cantidad :

Muestra la cantidad = 0 para los clientes que no tienen pedidos.

# SQL

## OUTER JOIN

**Recordatorio:** en SQL, los JOIN se evalúan **de izquierda a derecha**: primero se forma una tabla intermedia, en este caso entre *Cliente* y *Pedido*. Luego esa tabla intermedia se vuelve a unir con otra tabla, en el ejemplo *Detalle\_Pedido*, y así sucesivamente.

Listar todos los clientes y sus pedidos (si existen) con los detalles de los mismos (si el detalle existe)

```
SELECT c.nombre_empresa, p.id_pedido, dp.id_producto
FROM main. cliente c LEFT JOIN main.pedido p ON c.id_cliente = p.id_cliente
INNER JOIN main.detalle_pedido dp ON p.id_pedido = dp.id_pedido;
```

```
Query Editor Query History
1 SELECT c.nombre_empresa, p.id_pedido
2 FROM main. cliente c LEFT JOIN main.pedido p ON c.id_cliente = p.id_cliente
3 INNER JOIN main.detalle_pedido dp ON p.id_pedido = dp.id_pedido;
4
```

Data Output Explain Messages Notifications

|     | nombre_empresa                   | id_pedido |
|-----|----------------------------------|-----------|
| 812 | B's Beverages                    | 10578     |
| 814 | Lehmanns Marktstand              | 10593     |
| 815 | White Clover Markets             | 10596     |
| 816 | QUICK-Stop                       | 10515     |
| 817 | Seven Seas Imports               | 10523     |
| 818 | Lazy K Kountry Store             | 10545     |
| 819 | QUICK-Stop                       | 10451     |
| 820 | White Clover Markets             | 10483     |
| 821 | Gourmet Lanchonetes              | 10423     |
| 822 | Piccolo und mehr                 | 10427     |
| 823 | Princesa Isabel Vinhos           | 10433     |
| 824 | Hungry Owl All-Night Grocers     | 10380     |
| 825 | Berglunds snabbköp               | 10280     |
| 826 | Suprêmes délices                 | 10302     |
| 827 | Hungry Owl All-Night Grocers     | 10309     |
| 828 | Wartian Herkku                   | 10320     |
| 829 | Folk och få HB                   | 10264     |
| 830 | Split Rail Beer & Ale            | 10271     |
| 831 | Paris spécialités                | [null]    |
| 832 | FISSA Fabrica Inter. Salchich... | [null]    |

Tabla intermedia de la primera reunión (left join), muestra con NULLI datos de pedidos para clientes inactivos

```
Query Editor Query History
1 SELECT c.nombre_empresa, p.id_pedido, dp.id_producto
2 FROM main. cliente c LEFT JOIN main.pedido p ON c.id_cliente = p.id_cliente
3 INNER JOIN main.detalle_pedido dp ON p.id_pedido = dp.id_pedido;
4
```

Data Output Explain Messages Notifications

|      | nombre_empresa             | id_pedido | id_producto |
|------|----------------------------|-----------|-------------|
| 2133 | Rattlesnake Canyon Grocery | 11077     | 7           |
| 2136 | Rattlesnake Canyon Grocery | 11077     | 8           |
| 2137 | Rattlesnake Canyon Grocery | 11077     | 10          |
| 2138 | Rattlesnake Canyon Grocery | 11077     | 12          |
| 2139 | Rattlesnake Canyon Grocery | 11077     | 13          |
| 2140 | Rattlesnake Canyon Grocery | 11077     | 14          |
| 2141 | Rattlesnake Canyon Grocery | 11077     | 16          |
| 2142 | Rattlesnake Canyon Grocery | 11077     | 20          |
| 2143 | Rattlesnake Canyon Grocery | 11077     | 23          |
| 2144 | Rattlesnake Canyon Grocery | 11077     | 32          |
| 2145 | Rattlesnake Canyon Grocery | 11077     | 39          |
| 2146 | Rattlesnake Canyon Grocery | 11077     | 41          |
| 2147 | Rattlesnake Canyon Grocery | 11077     | 46          |
| 2148 | Rattlesnake Canyon Grocery | 11077     | 52          |
| 2149 | Rattlesnake Canyon Grocery | 11077     | 55          |
| 2150 | Rattlesnake Canyon Grocery | 11077     | 60          |
| 2151 | Rattlesnake Canyon Grocery | 11077     | 64          |
| 2152 | Rattlesnake Canyon Grocery | 11077     | 66          |
| 2153 | Rattlesnake Canyon Grocery | 11077     | 73          |
| 2154 | Rattlesnake Canyon Grocery | 11077     | 75          |
| 2155 | Rattlesnake Canyon Grocery | 11077     | 77          |

Tabla de la segunda reunión (inner join), entre la intermedia y Detalle\_pedido.

¿La consulta mostrará todos los clientes?

¿Qué ocurre con los clientes que no tienen pedidos?

¿Qué ocurre con los clientes que tienen pedidos pero sin detalle?

¿El LEFT JOIN está cumpliendo su propósito? Justifique.

# SQL

## OUTER JOIN

Listar todos los clientes y sus pedidos (si existen) con los detalles de los mismos (si el detalle existe)

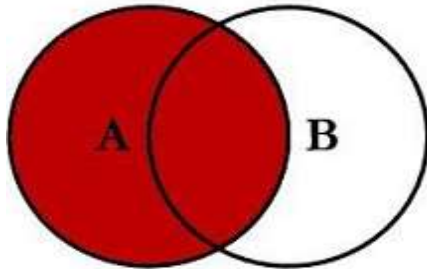
```
SELECT c.nombre_empresa, p.id_pedido, dp.id_producto
FROM main.cliente c LEFT JOIN main.pedido p ON c.id_cliente = p.id_cliente
LEFT JOIN main.detalle_pedido dp ON p.id_pedido = dp.id_pedido;
```

```
1 SELECT c.id_cliente, c.nombre_empresa, p.id_pedido, dp.id_producto
2 FROM main.cliente c LEFT JOIN main.pedido p ON c.id_cliente = p.id_cliente
3 LEFT JOIN main.detalle_pedido dp ON p.id_pedido = dp.id_pedido;
4
```

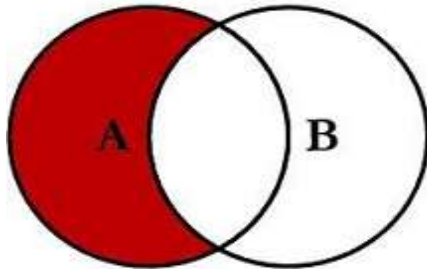
|      | Data Output                         | Explain                                  | Messages             | Notifications          |
|------|-------------------------------------|------------------------------------------|----------------------|------------------------|
|      | id_cliente<br>character varying (5) | nombre_empresa<br>character varying (60) | id_pedido<br>integer | id_producto<br>integer |
| 2137 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 10                     |
| 2138 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 12                     |
| 2139 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 13                     |
| 2140 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 14                     |
| 2141 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 16                     |
| 2142 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 20                     |
| 2143 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 23                     |
| 2144 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 32                     |
| 2145 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 39                     |
| 2146 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 41                     |
| 2147 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 46                     |
| 2148 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 52                     |
| 2149 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 55                     |
| 2150 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 60                     |
| 2151 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 64                     |
| 2152 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 66                     |
| 2153 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 73                     |
| 2154 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 75                     |
| 2155 | RATTC                               | Rattlesnake Canyon Grocery               | 11077                | 77                     |
| 2156 | PARIS                               | Paris spécialités                        | 20000                | [null]                 |
| 2157 | FISSA                               | FISSA Fabrica Inter. Salchich...         | [null]               | [null]                 |

Así muestra clientes sin pedidos como FISSA y cliente con pedido pero sin detalle para el pedido, el caso de PARIS.

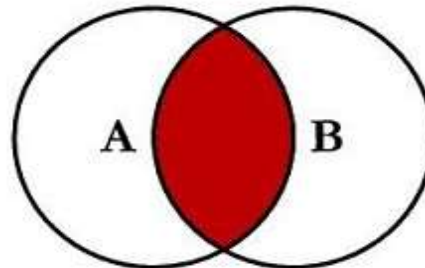
# SQL JOINS



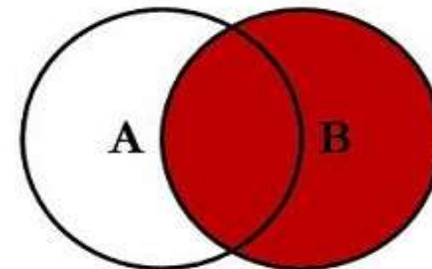
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



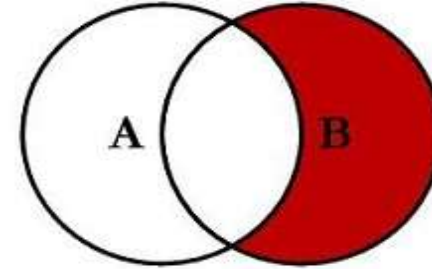
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```



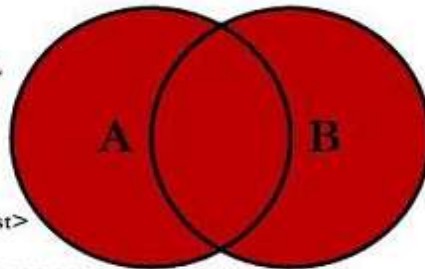
```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



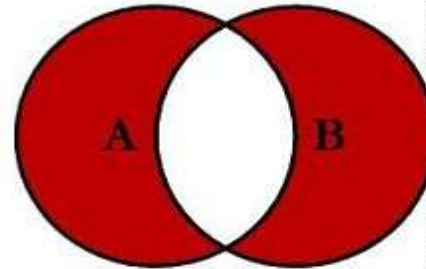
```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```